

Turing-Maschinen und Co

(Turing Machines and Friends)

Andreas Schäfer

Theoretische Informatik

TH Lübeck

Gibt das Programm „Hello World“ aus?

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    puts("Hello World\n"); /*prints Hello World*/
    return EXIT_SUCCESS;
}
```

Gibt das Programm „Hello World“ aus?

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    int total = 3;
    int x, y, z;
    while (1) {
        for (x = 1; x < total - 2; x++) {
            for (y = 1; y < total - x - 1; y++) {
                z = total - x - y;
                if (x * x * x + y * y * y == z * z * z) {
                    puts("Hello World\n");
                    return EXIT_SUCCESS;
                }
            }
        }
        total++;
    }
}
```

Gibt das Programm „Hello World“ aus?

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    int total = 3; // Summe x+y+z
    int x, y, z;
    while (1) {
        for (x = 1; x < total - 2; x++) { // - 2 weil y und z >= 1
            for (y = 1; y < total - x - 1; y++) { // -1 weil z >= 1
                z = total - x - y;
                if (x * x * x + y * y * y == z * z * z) { // Fermat
                    puts("Hello World\n");
                    return EXIT_SUCCESS;
                }
            }
        }
        total++;
    }
}
```

- Das dritte Programm gibt HelloWorld aus, wenn der Satz von Fermat nicht stimmt
- Die Frage war Jahrhunderte ungelöst

- Das dritte Programm gibt HelloWorld aus, wenn der Satz von Fermat nicht stimmt
- Die Frage war Jahrhunderte ungelöst
- Noch schlimmer: Das Programm gibt HelloWorld aus, weil bei 2048 ein Überlauf auftritt

- Das dritte Programm gibt HelloWorld aus, wenn der Satz von Fermat nicht stimmt
- Die Frage war Jahrhunderte ungelöst
- Noch schlimmer: Das Programm gibt HelloWorld aus, weil bei 2048 ein Überlauf auftritt
- Implementierungsdetails: Fortgeschrittene Themen :)

FRAGE: WAS KÖNNEN COMPUTER (EFFIZIENT)

Computer



```
public class Quicksort {  
    // requires 0 <= lo;  
    // requires hi < a.length;  
    // ensures { forall int k, n: lo <= n && n < k && k <= hi : a[n] == a[k];  
    public static void quicksort(int[] a, int lo, int hi) {  
        if (hi <= lo)  
            return;  
        int j = partition(a, lo, hi);  
        quicksort(a, lo, j - 1);  
        quicksort(a, j + 1, hi);  
    }  
    // requires 0 <= lo && lo < a.length;  
    // requires 0 <= hi && hi < a.length;  
    // requires lo < hi;  
    // ensures 0 <= result && (result < a.length;
```

Programm

101101110101

Daten

Fragen

1. Welche Probleme kann man überhaupt lösen?
2. Welche Probleme kann man effizient lösen?

FRAGE: WAS KÖNNEN COMPUTER (EFFIZIENT)

Computer



```
public class Quicksort {  
    // requires 0 <= lo;  
    // requires hi < a.length;  
    // ensures {forall let k,n: lo <= n && n < k && k <= hi :a[n] <= a[k]}  
    public static void quicksort(int[] a, int lo, int hi) {  
        if (hi <= lo)  
            return;  
        int j = partition(a, lo, hi);  
        quicksort(a, lo, j - 1);  
        quicksort(a, j + 1, hi);  
    }  
    // requires 0 <= lo && lo < a.length;  
    // requires 0 <= hi && hi < a.length;  
    // requires lo < hi;  
    // ensures 0 <= result && result < a.length;  
}
```

Programm

101101110101

Daten

Fragen

1. Welche Probleme kann man überhaupt lösen?
2. Welche Probleme kann man effizient lösen?

Anwendung

Eigenschaften von Programmen:

- Halten
- Erreichbarer Code

FRAGE: WAS KÖNNEN COMPUTER (EFFIZIENT)

Computer



```
public class QuickSort {  
    // requires 0 <= lo;  
    // requires hi < a.length;  
    // ensures {forall int k, n: lo <= n && n < k && k <= hi : a[n] == a[k]}  
    public static void quicksort(int[] a, int lo, int hi) {  
        if (hi <= lo)  
            return;  
        int j = partition(a, lo, hi);  
        quicksort(a, lo, j - 1);  
        quicksort(a, j + 1, hi);  
    }  
    // requires 0 <= lo && lo < a.length;  
    // requires 0 <= hi && hi < a.length;  
    // requires lo < hi;  
    // ensures 0 <= result && (result < a.length;
```

Programm

101101110101

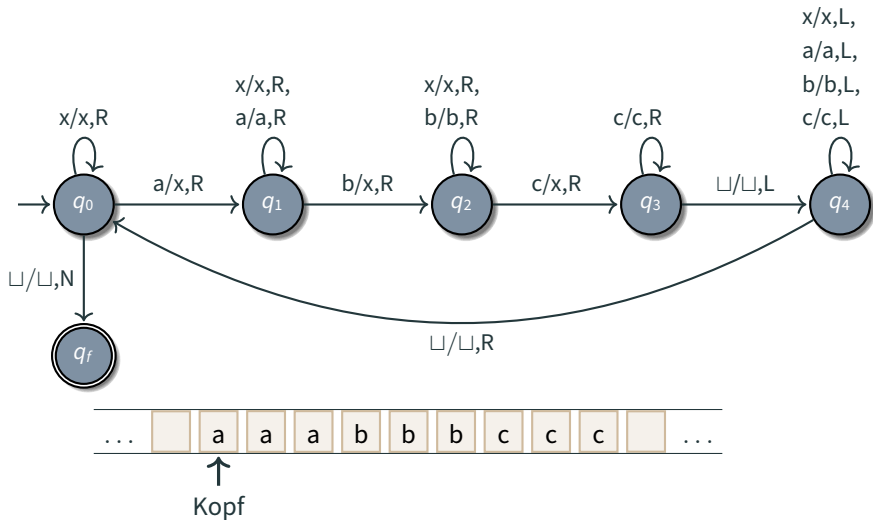
Daten

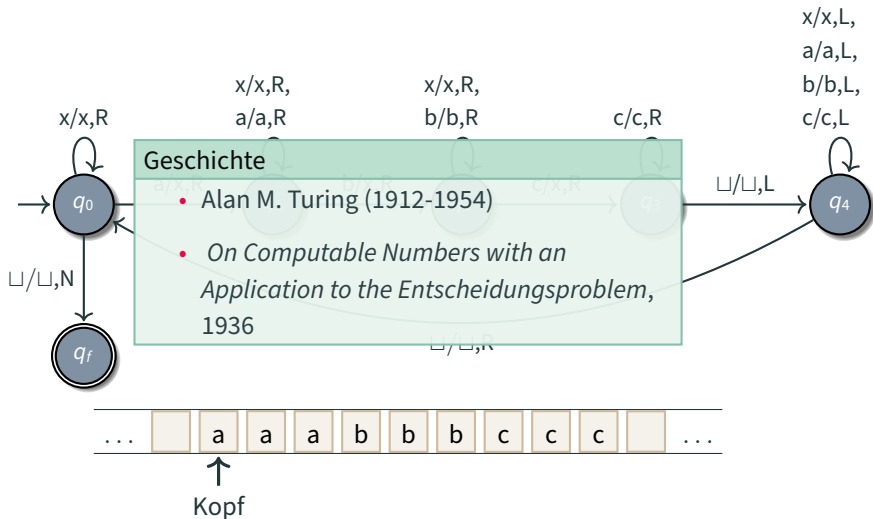
Fragen

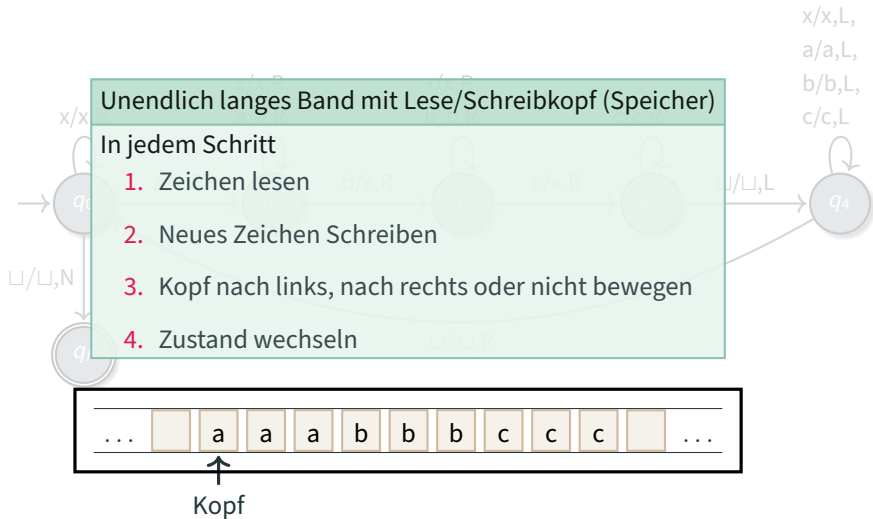
1. Welche Probleme kann man überhaupt lösen?
2. Welche Probleme kann man effizient lösen?

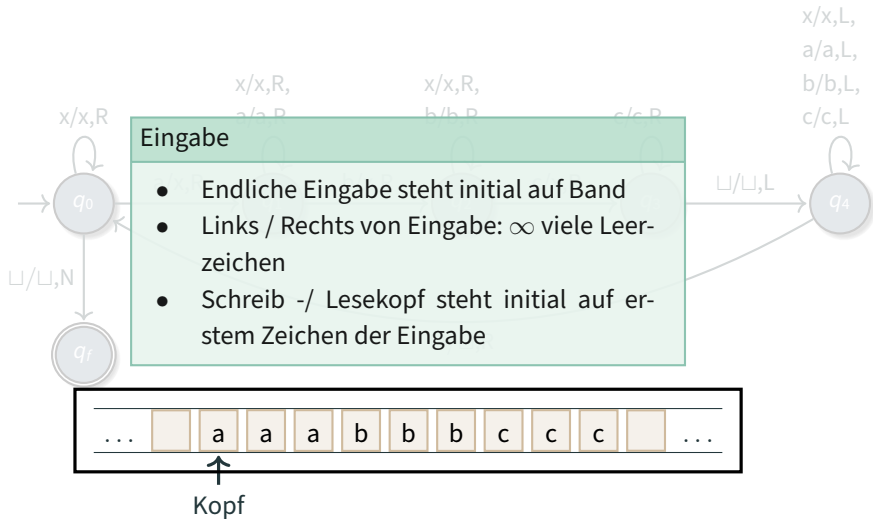
Maschinenmodell

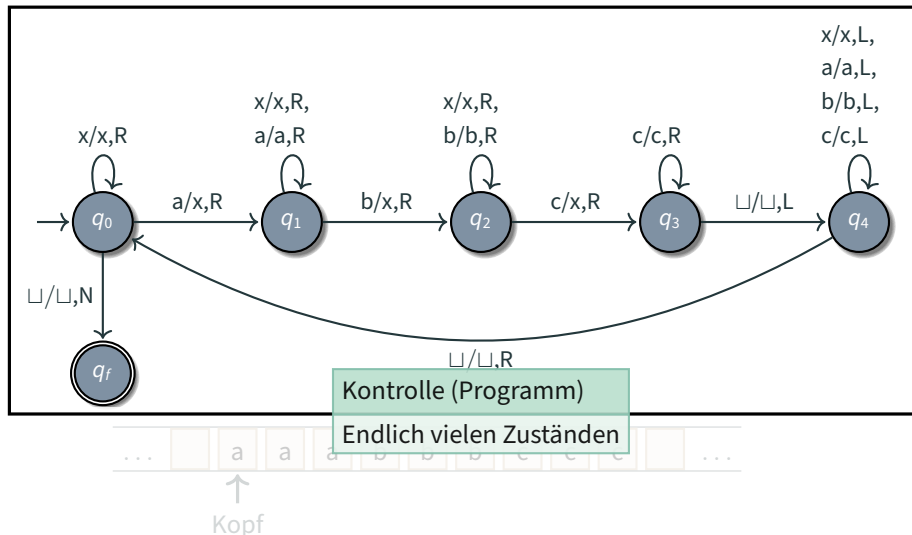
- Einfach formal zu beschreiben
- Kann alles was unsere Computer auch können

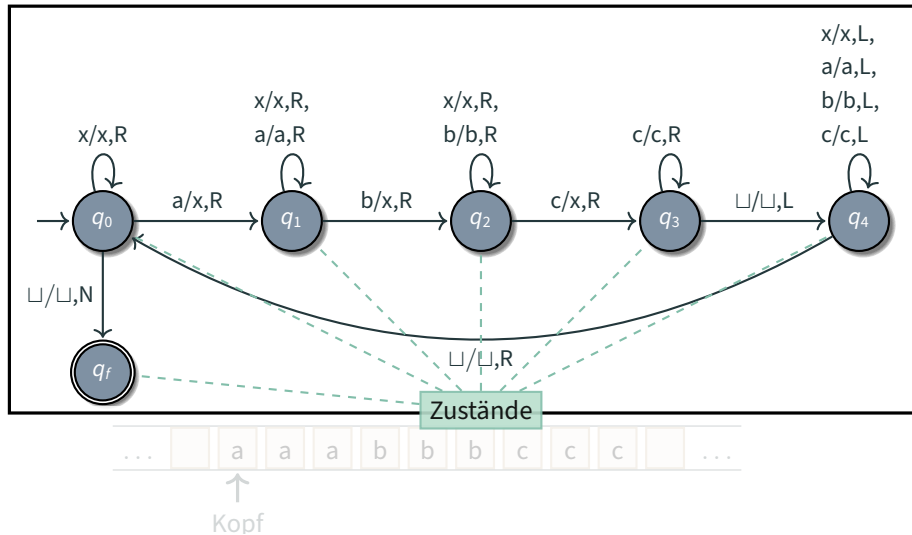


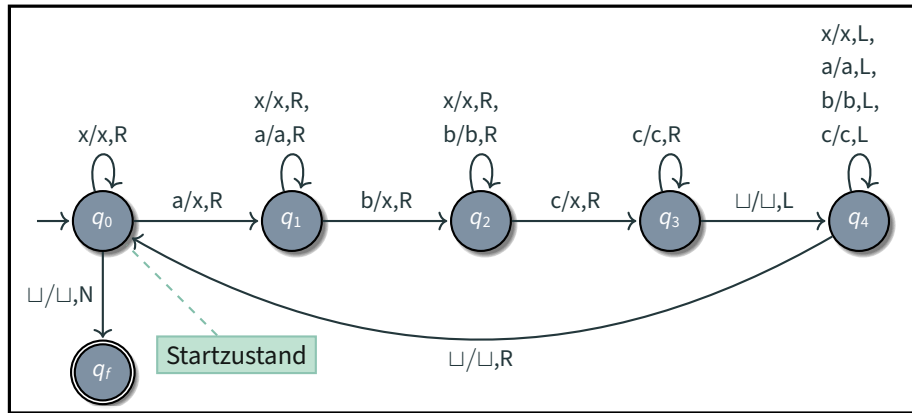


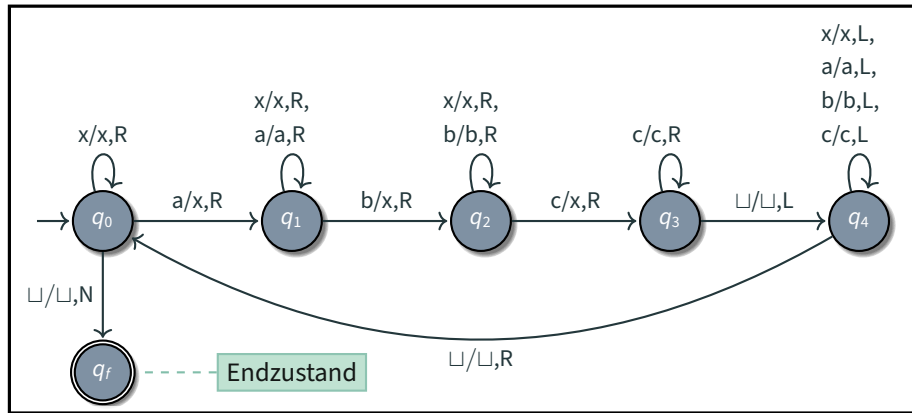


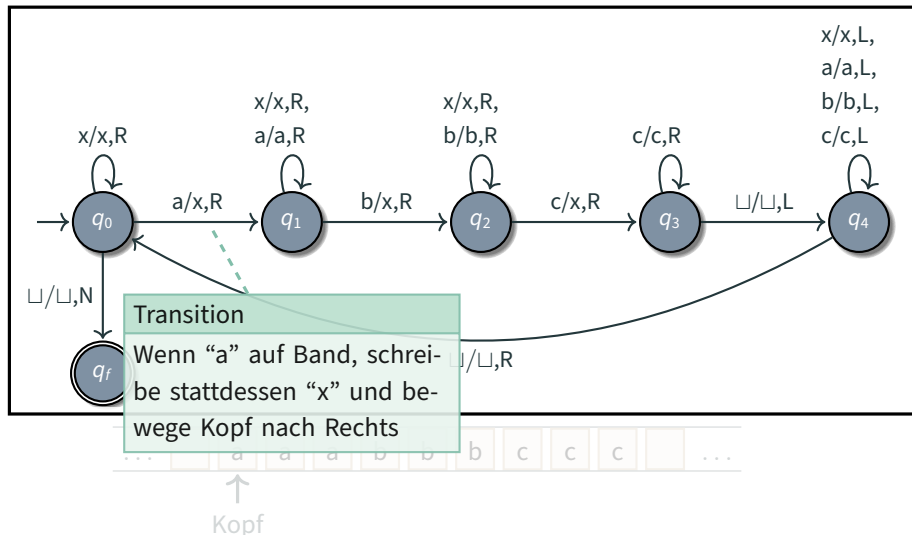


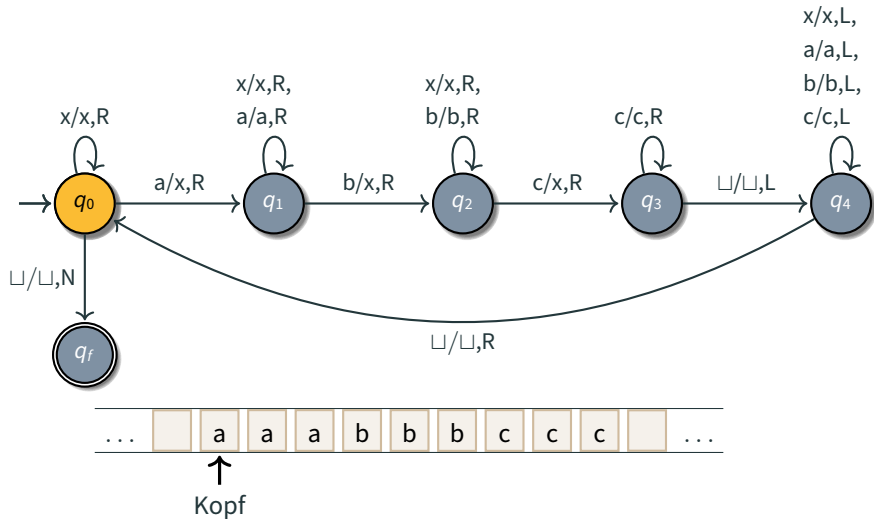


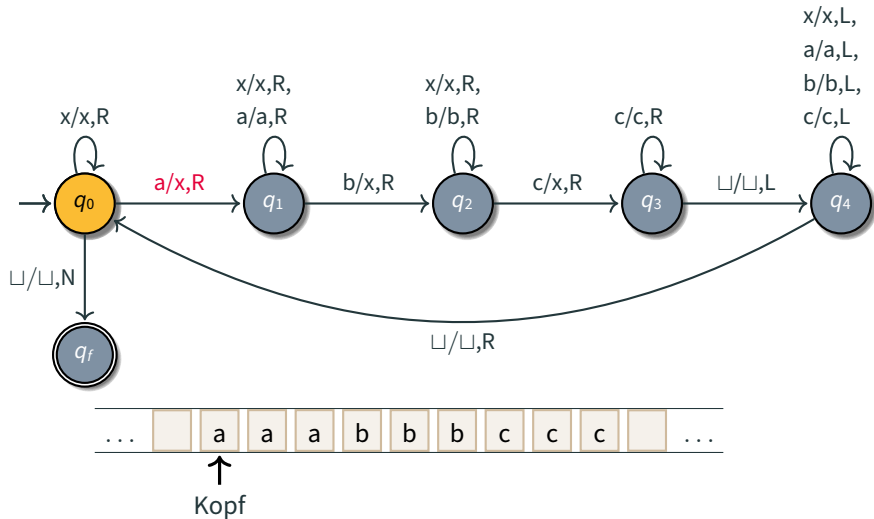


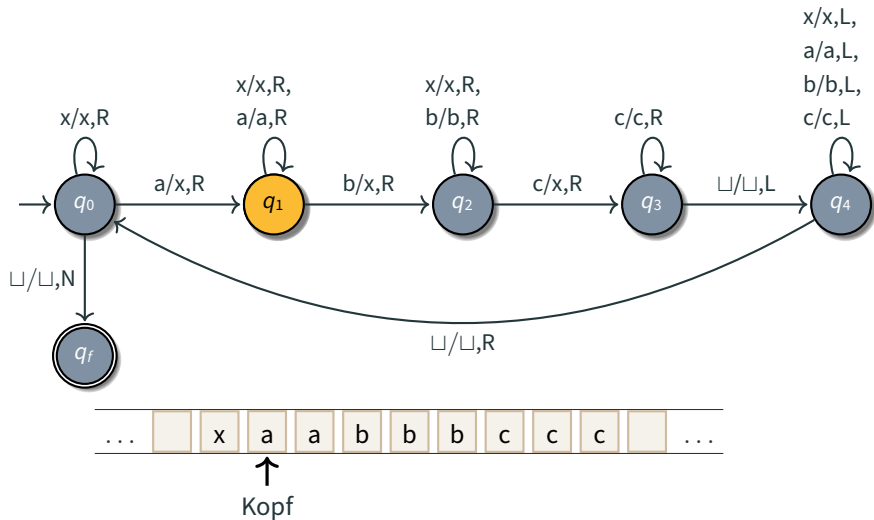


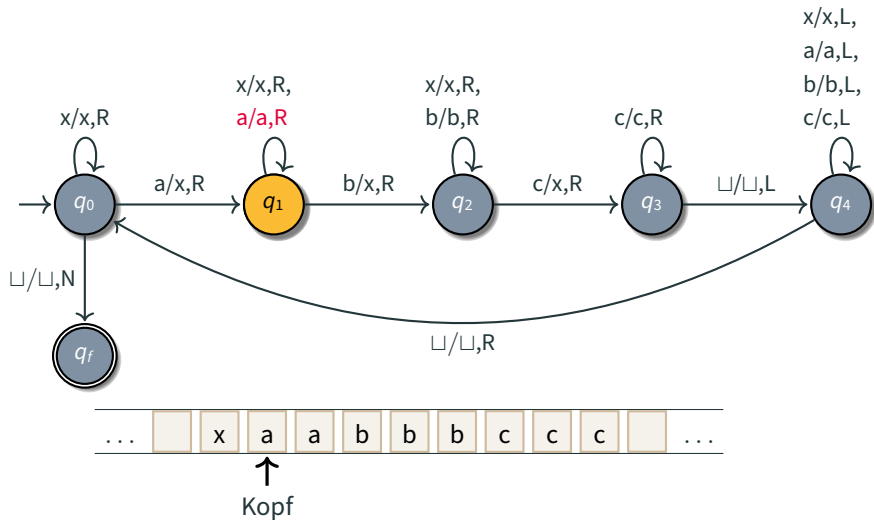


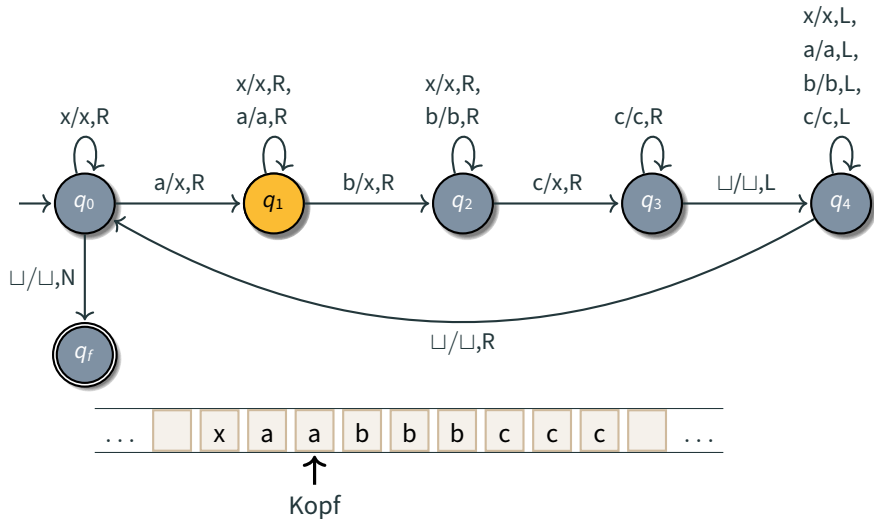


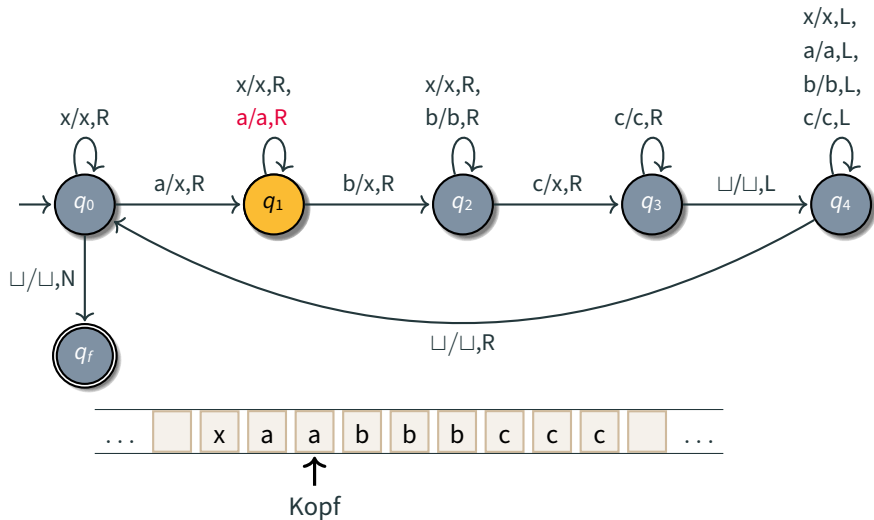


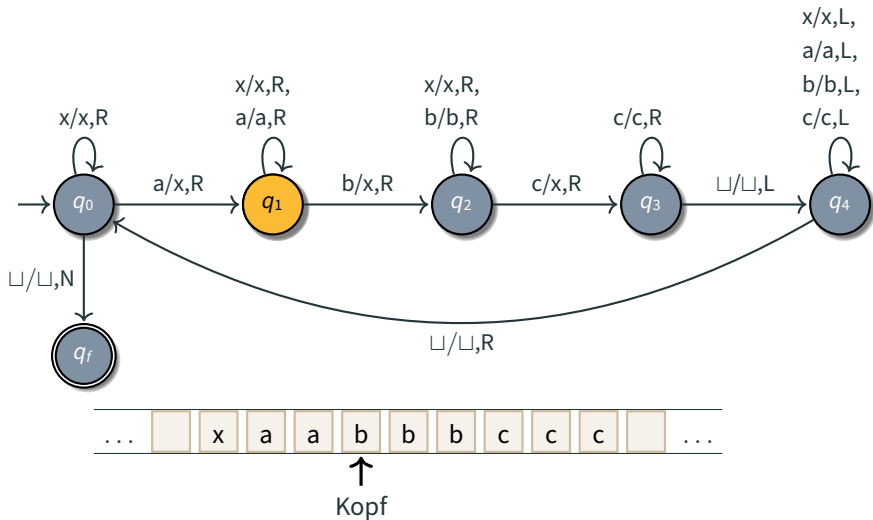


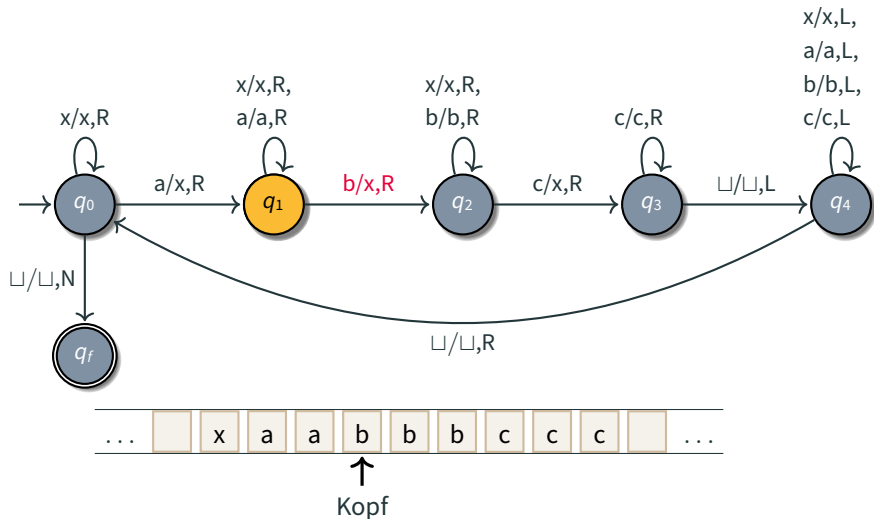


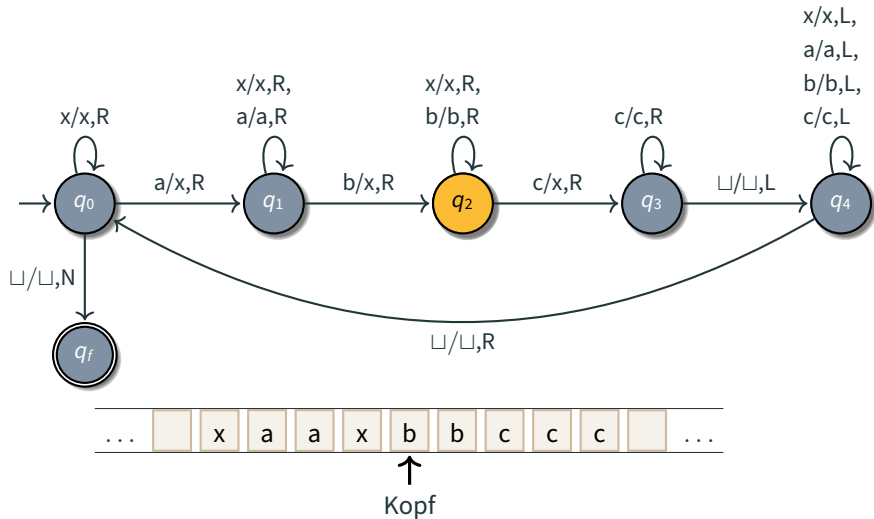


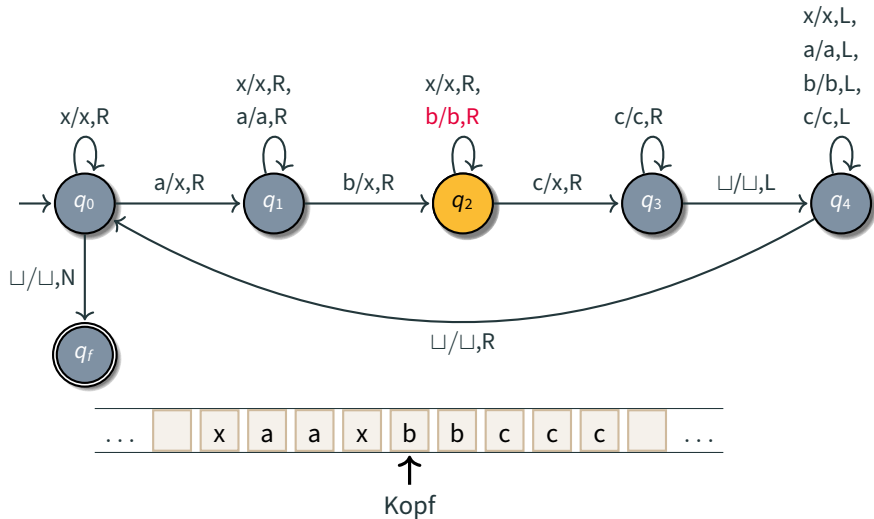


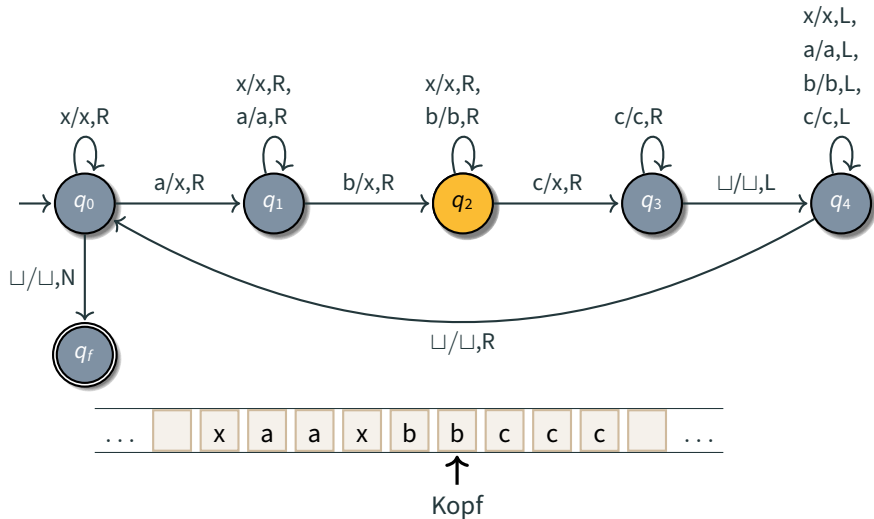


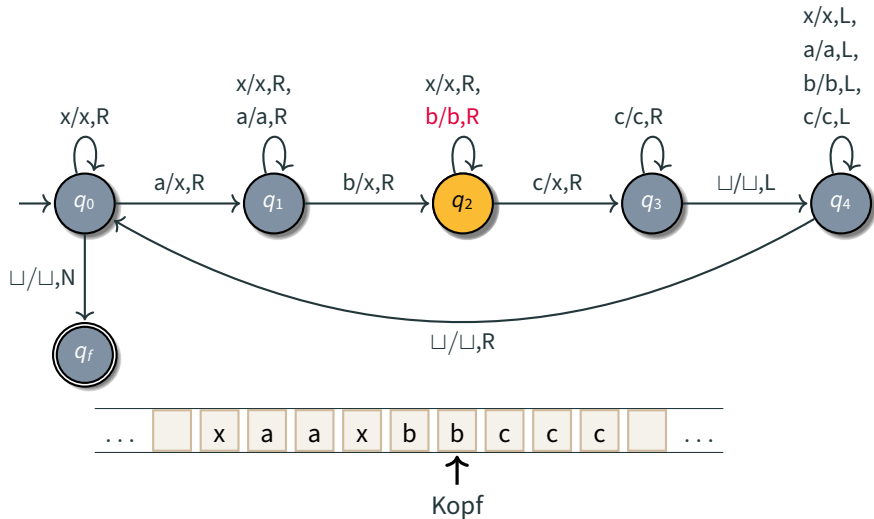


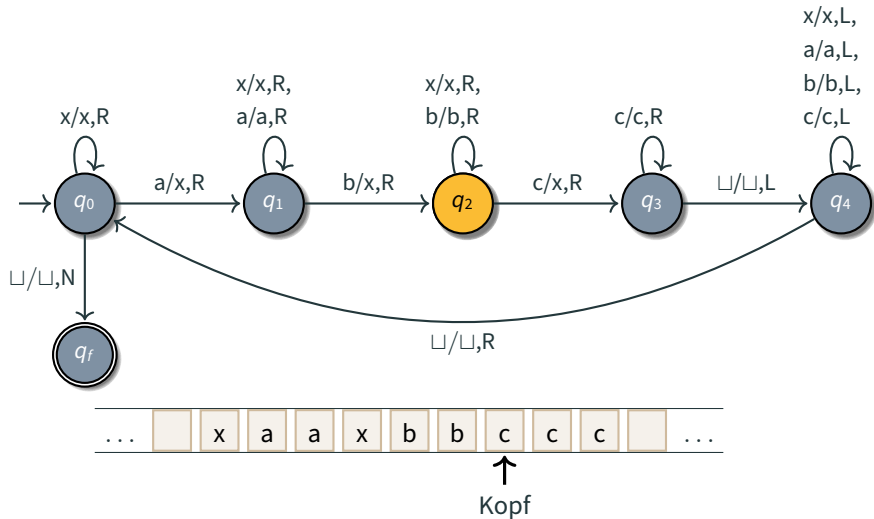


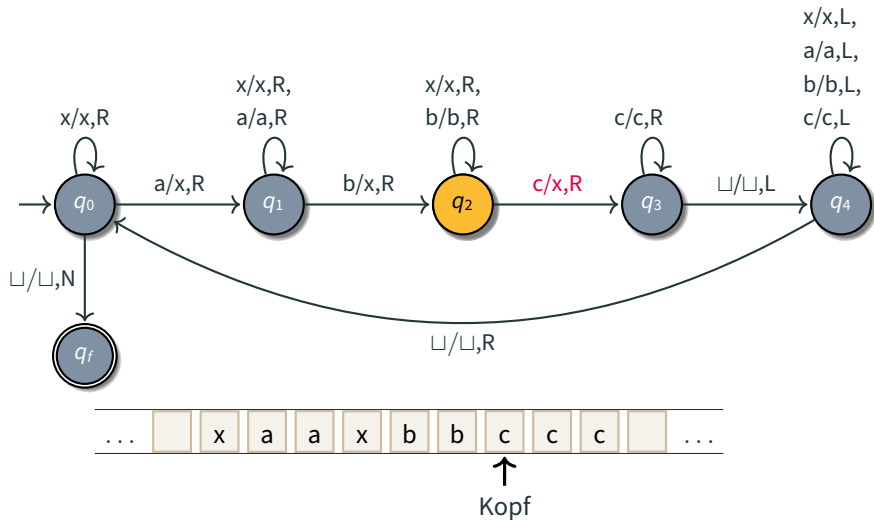


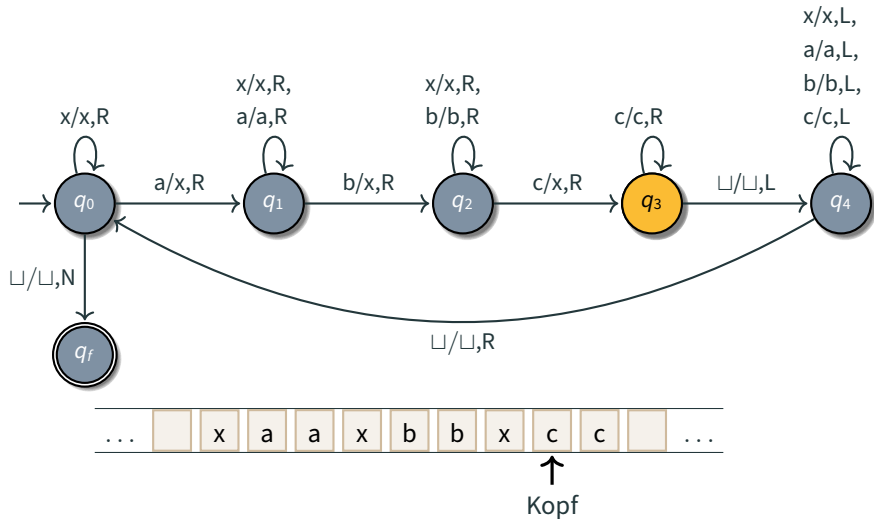


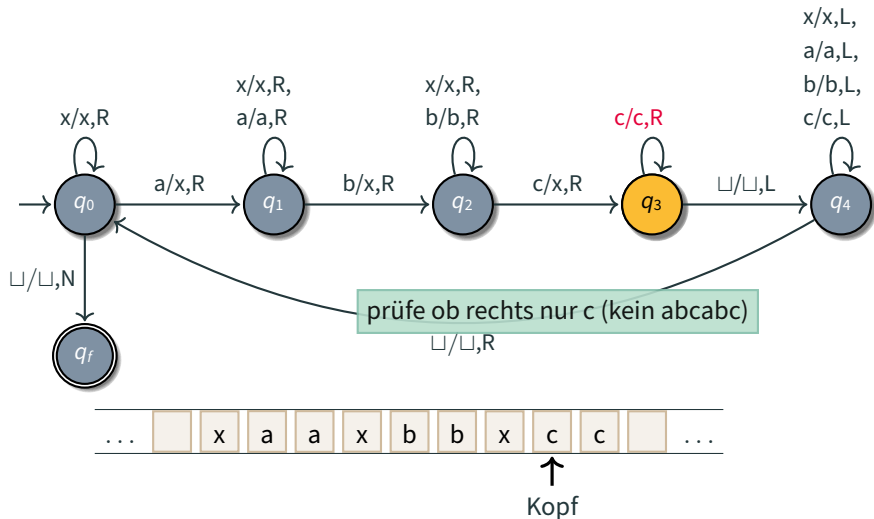


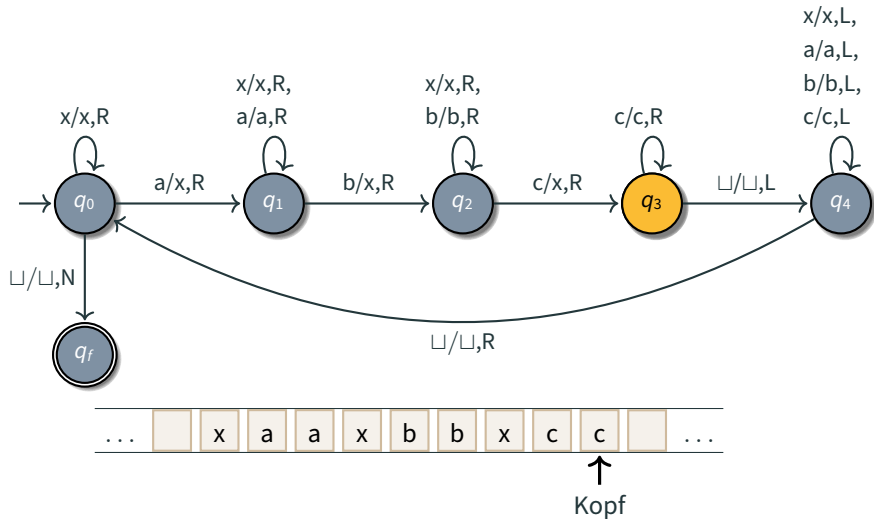


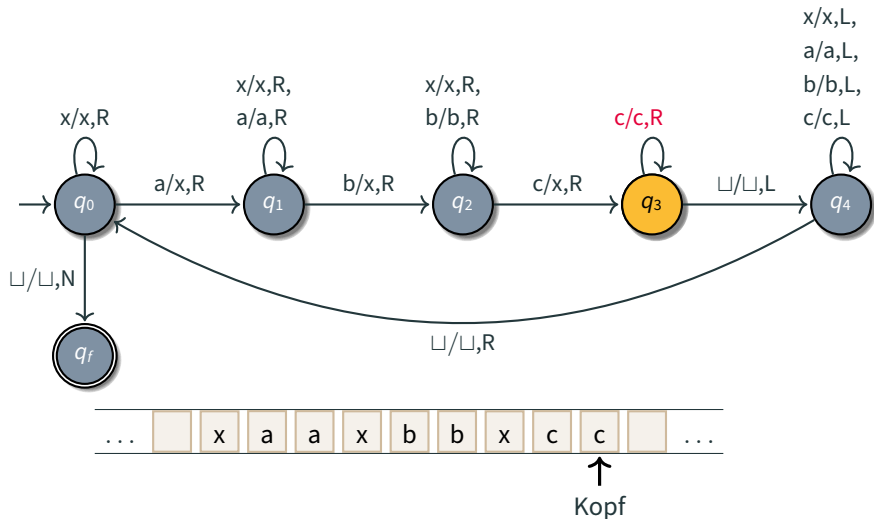


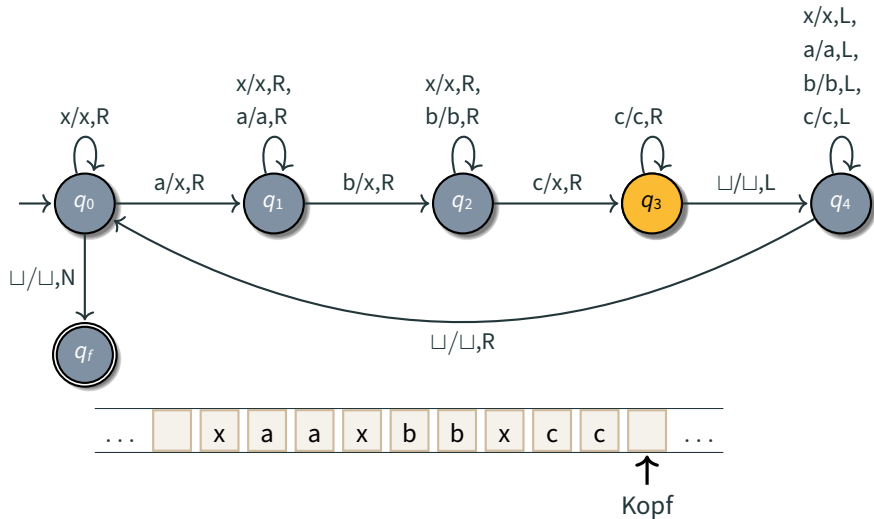


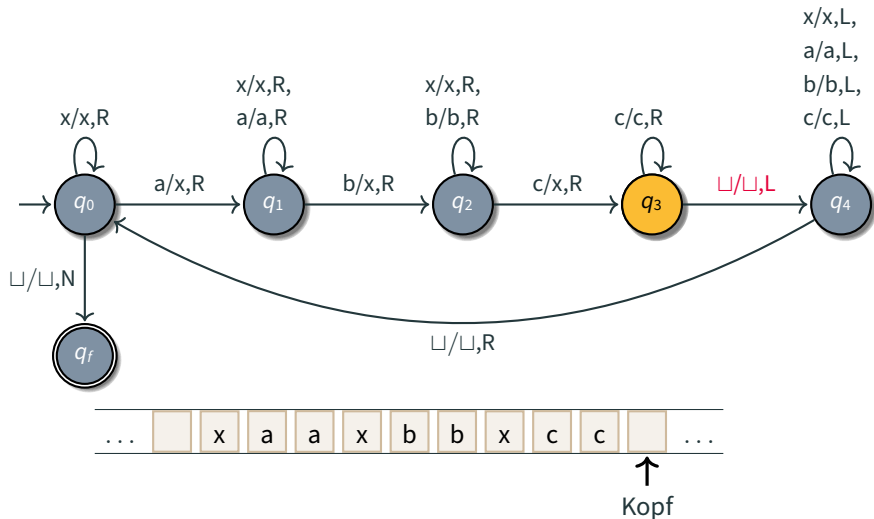


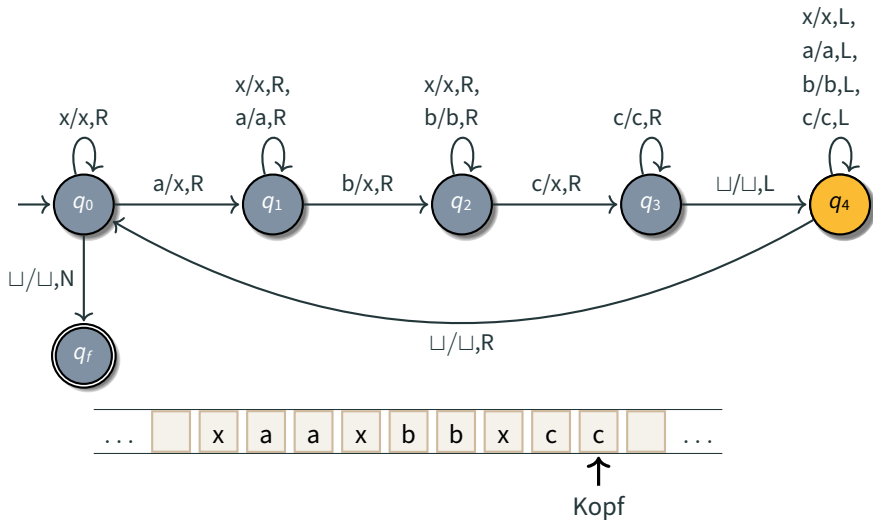


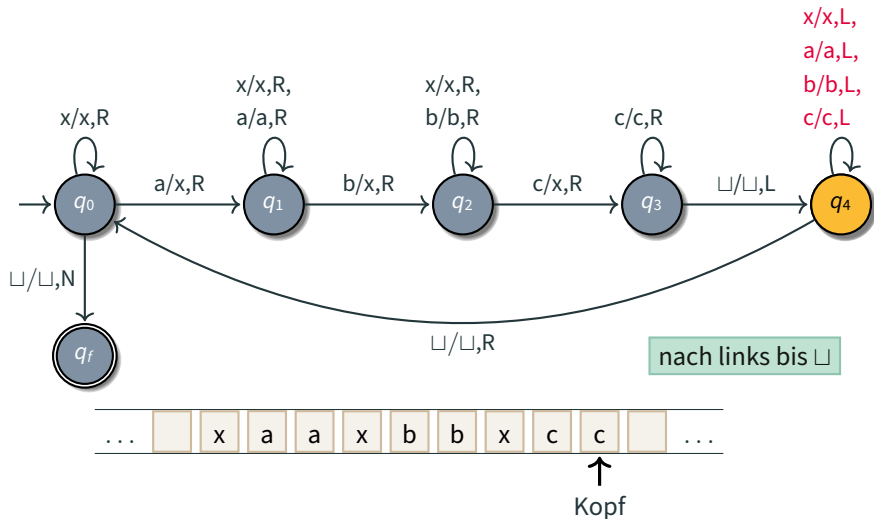


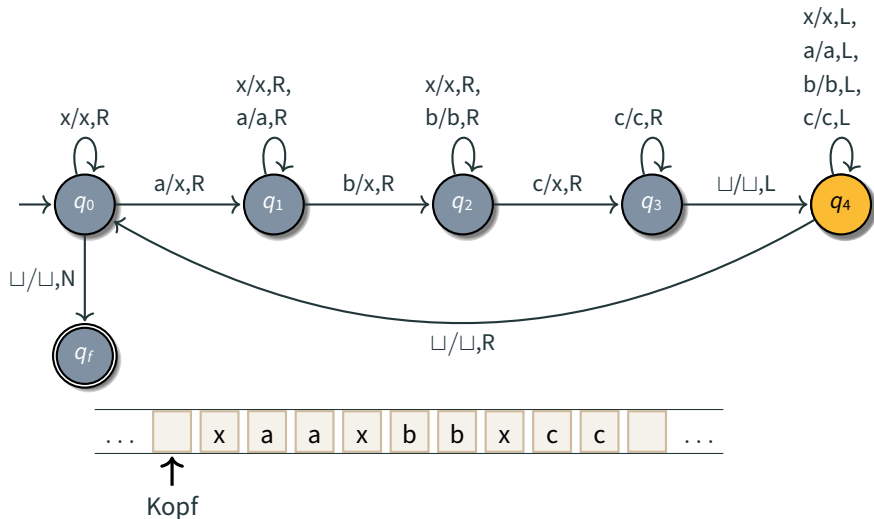


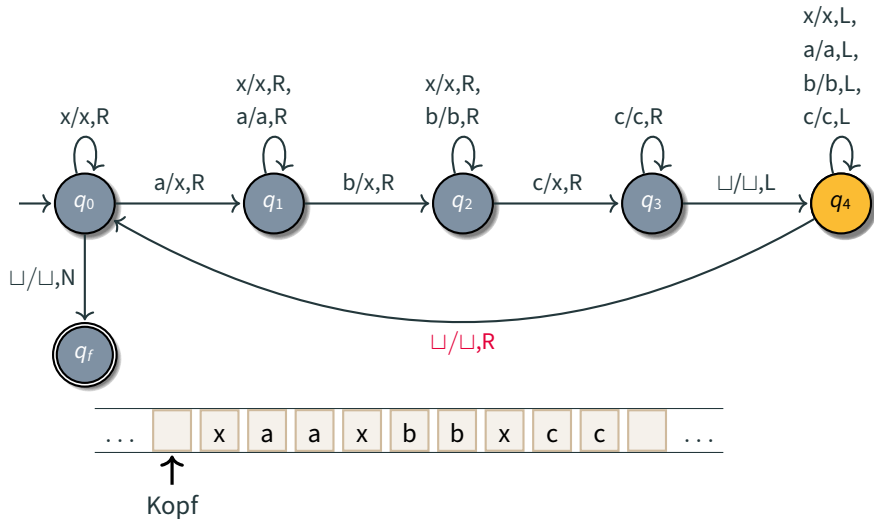


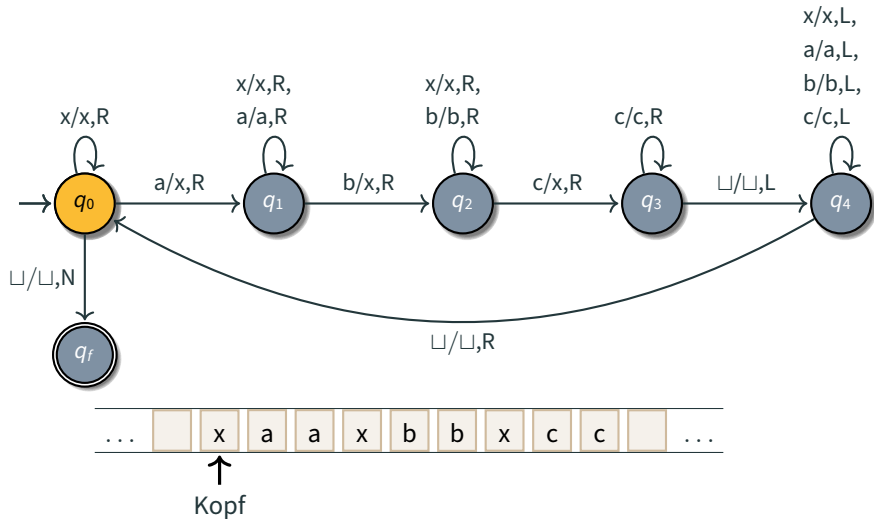


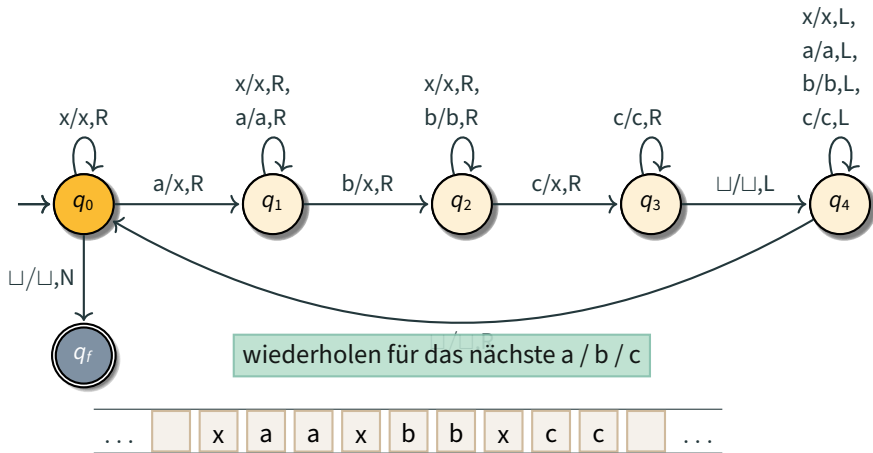


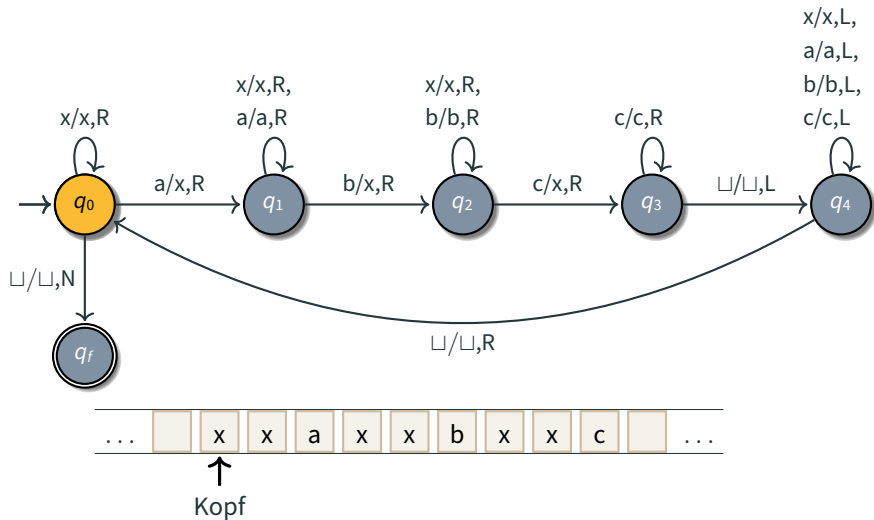


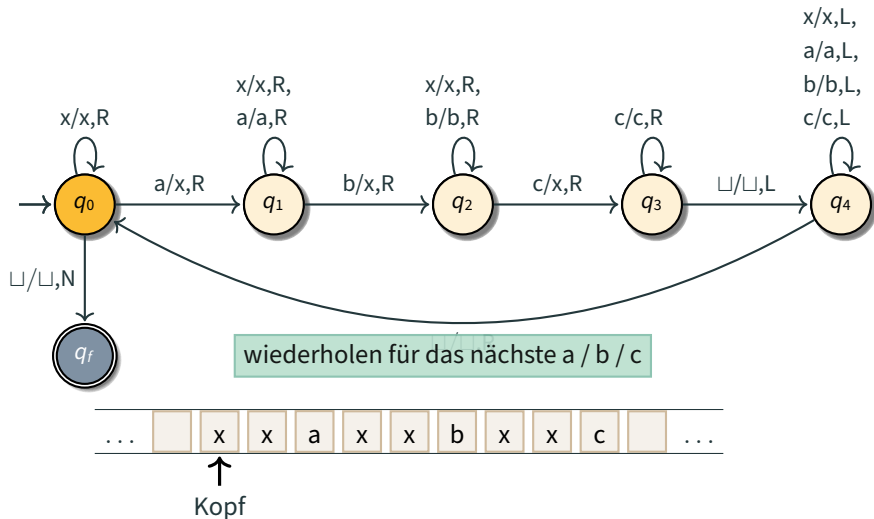


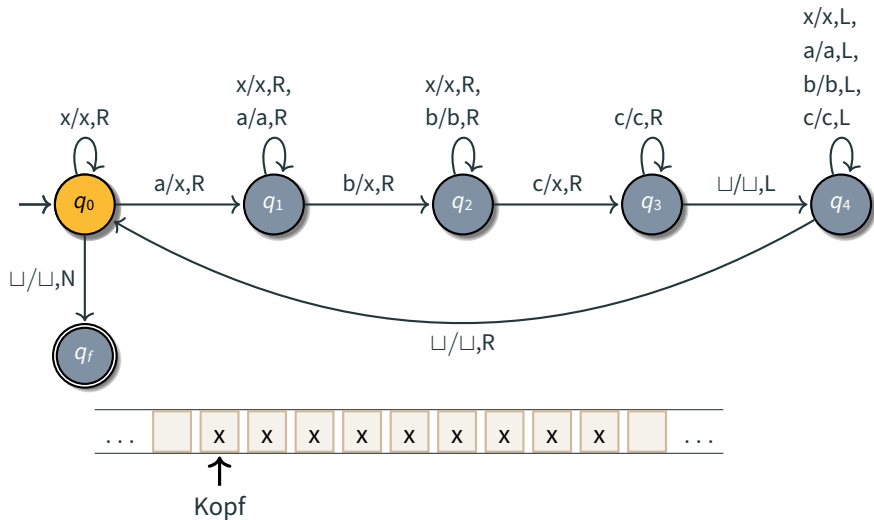


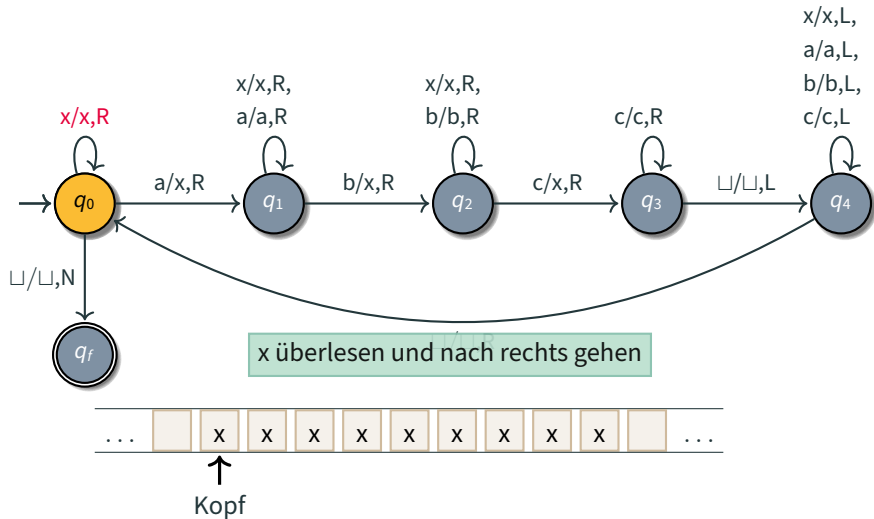


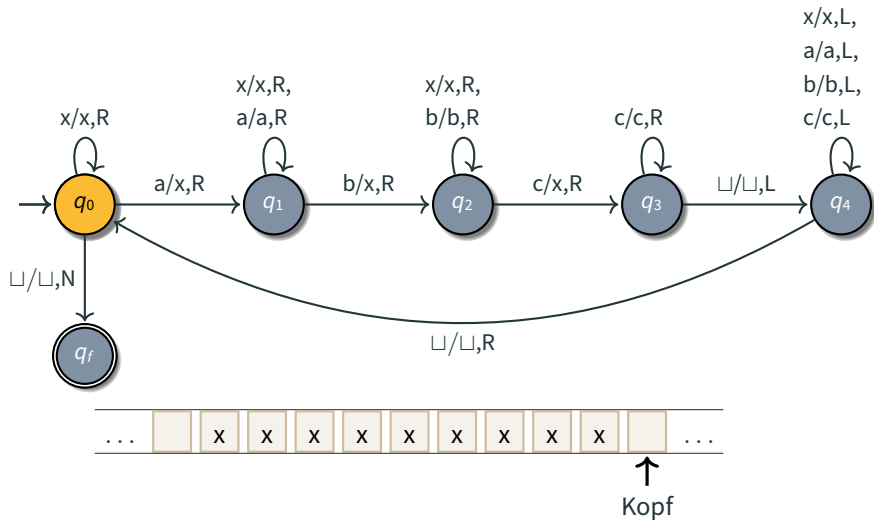


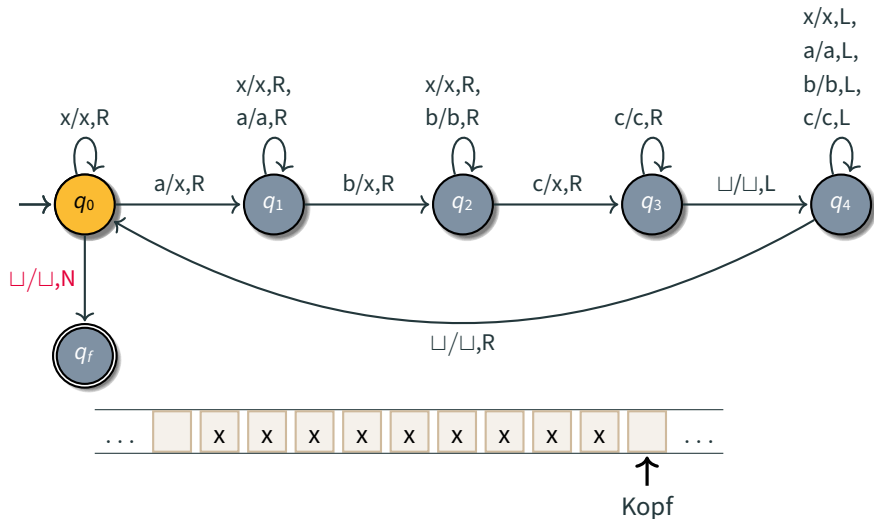


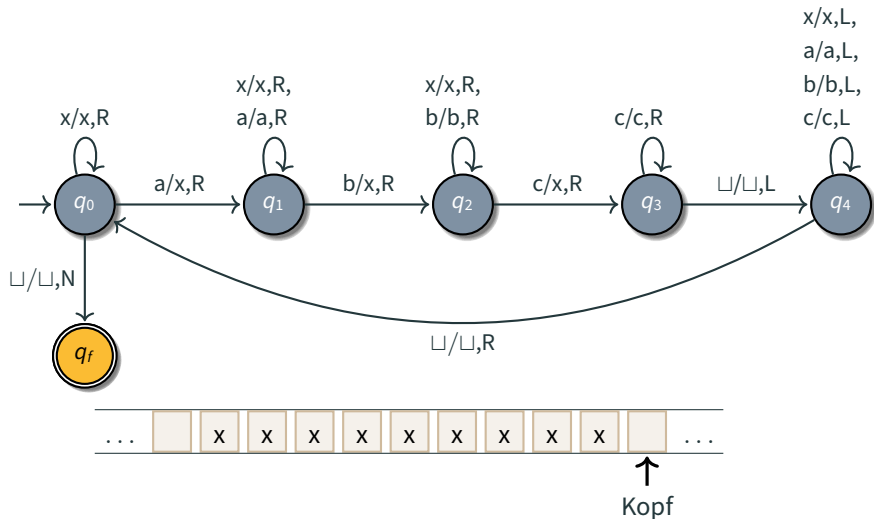












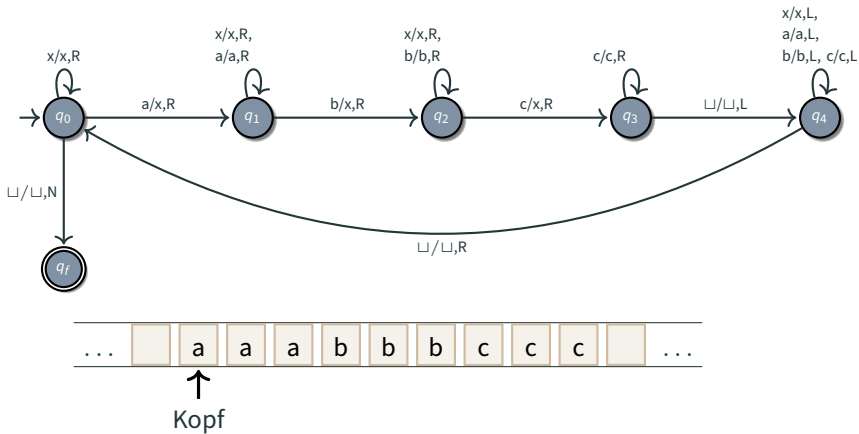


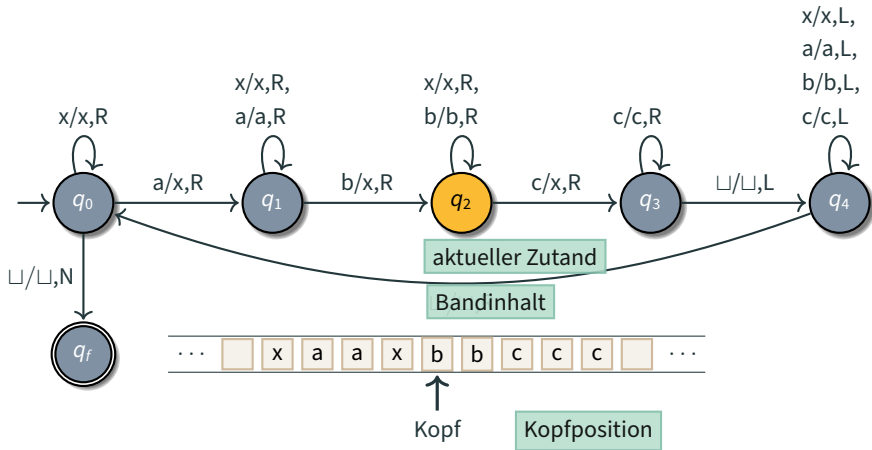
Abbildung 1: Turing-Maschine akzeptiert $a^n b^n c^n$ – eine nicht kfr. Sprache

Definition (Turing-Maschine)

Eine (deterministische) Turing-Maschine (Turing Machine, TM) ist ein 7-Tupel $M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$ mit

- Zustandsmenge Q
- Eingabealphabet Σ
- Bandalphabet $\Gamma \supset \Sigma$
- Transitionsfunktion $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, N\}$ (L : Links, R : Rechts, N : Neutral – nicht bewegen)
- Startzustand q_0
- $\sqcup \in \Gamma$ das Blank (Leerzeichen) (Kein Eingabezeichen: $\sqcup \notin \Sigma$)
- F eine Menge von Endzuständen

TURING-MASCHINEN KONFIGURATION



Kurz als: $xaaxq_2bbccc$

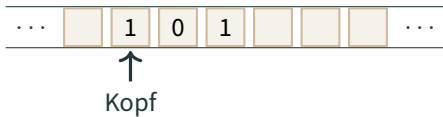
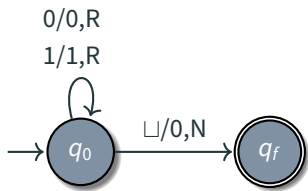
- Eine Turing-Maschine *akzeptiert* ein Wort, wenn die Turing-Maschine angesetzt auf das Wort in einen Endzustand kommen kann
- Kurz: $(q_0w) \rightarrow^* (\alpha q_f \beta), q_f \in F$

- Eine Turing-Maschine *akzeptiert* ein Wort, wenn die Turing-Maschine angesetzt auf das Wort in einen Endzustand kommen kann
- Kurz: $(q_0w) \rightarrow^* (\alpha q_f \beta), q_f \in F$

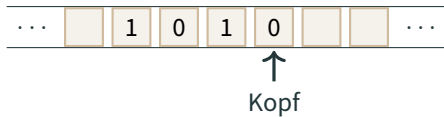
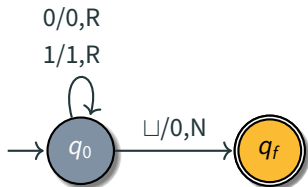
Akzeptieren $\neq$ Entscheiden

Hier wurde Akzeptieren von Sprachen definiert. Später kommt noch *Entscheiden* von Sprachen durch Turing-Maschinen, das meint etwas anderes.

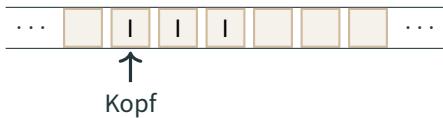
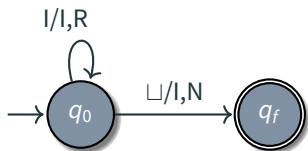
- Eine Turing-Maschine berechnet eine Funktion, wenn sie angesetzt auf die Kodierung des Eingabewertes die Kodierung des Funktionswerts auf das Band schreibt.
- $(q_0 w) \rightarrow^* (\alpha q_f \beta)$, $q_f \in F$ wobei w die Codierung des Eingabewertes ist und $\alpha\beta$ die Codierung des Funktionswerts ist.
- Beispielcodierungen: Binärcodierung, Unärcodierung (Bierdeckelnotation)



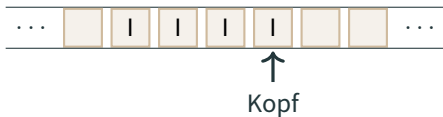
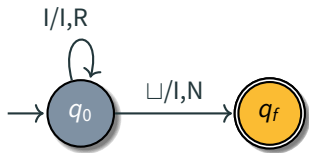
Was berechnet die TM?



Berechnet $f : \mathbb{N} \rightarrow \mathbb{N}, n \mapsto 2n$ mit Binärcodierung



Und diese TM?



Berechnet $f : \mathbb{N} \rightarrow \mathbb{N}, n \mapsto n + 1$ mit Unärcodierung (Bierdeckelnotation)

Aufgabe

Konstruieren Sie eine Turing-Maschine, die

1. $+2$ in Unärcodierung
2. $+1$ in Binärcodierung berechnet

- Problem: Erfasst man mit Turing-Maschinen alles was intuitiv als berechenbar gelten soll?

Churchsche These

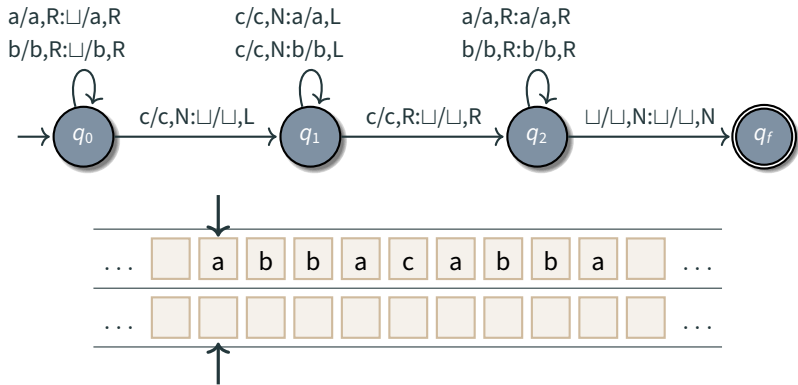
Die durch Turing-Maschinen berechenbaren Funktionen sind genau die im intuitiven Sinne berechenbaren Funktionen

Äquivalente Berechenbarkeitsmodelle sind

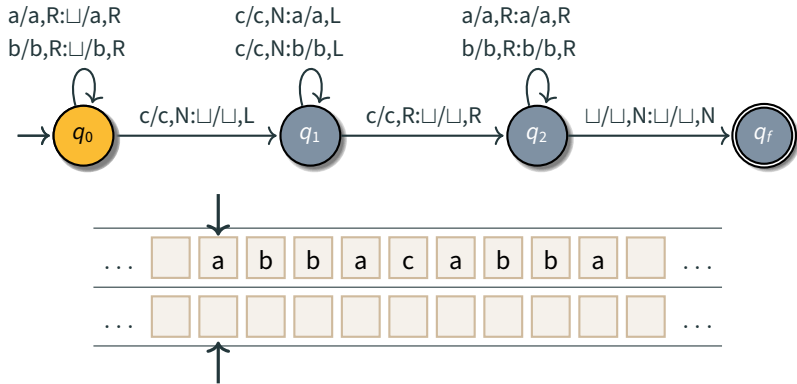
- GOTO Programme
- WHILE Programme
- μ -rekursive Funktionen
- ...

- Mehrbandturingmaschinen mit mehreren Bändern / Köpfen
- Nichtdeterministische Turing-Maschinen
- machen das Leben leichter, die Programme sind einfacher und kürzer
- können aber nicht mehr berechnen als normale TMs (!)
- sind aber vielleicht schneller

MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



q_0

Wort bis "c" auf 2.
Band kopieren

MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$

$a/a, R: \sqcup/a, R$
 $b/b, R: \sqcup/b, R$

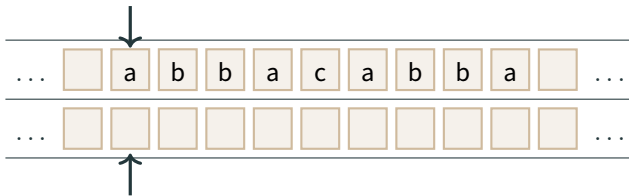
$c/c, N: a/a, L$
 $c/c, N: b/b, L$

$a/a, R: a/a, R$
 $b/b, R: b/b, R$

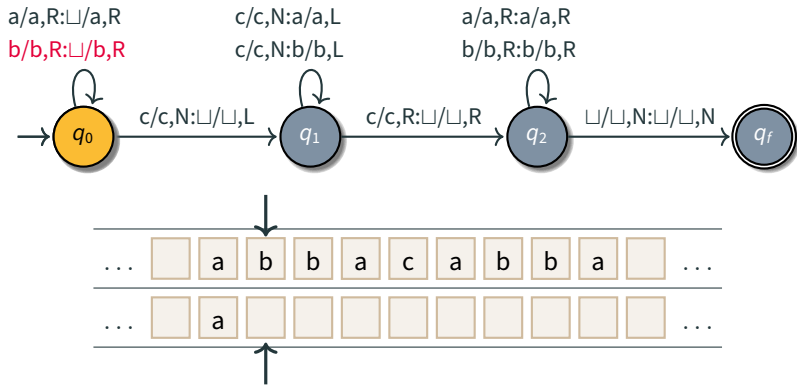


q_0

Wort bis "c" auf 2.
Band kopieren



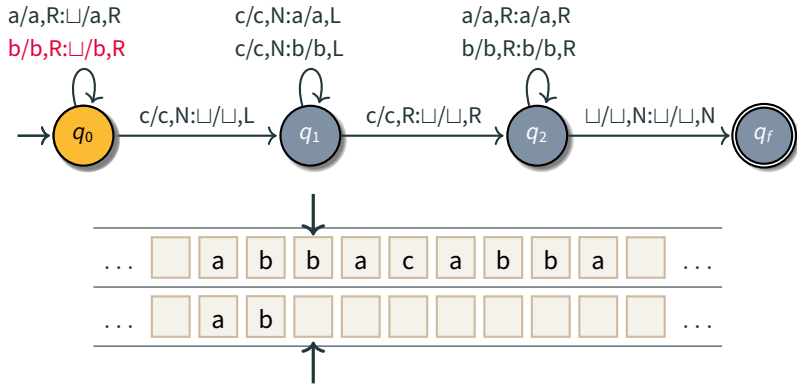
MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



q_0

Wort bis "c" auf 2.
Band kopieren

MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



q_0

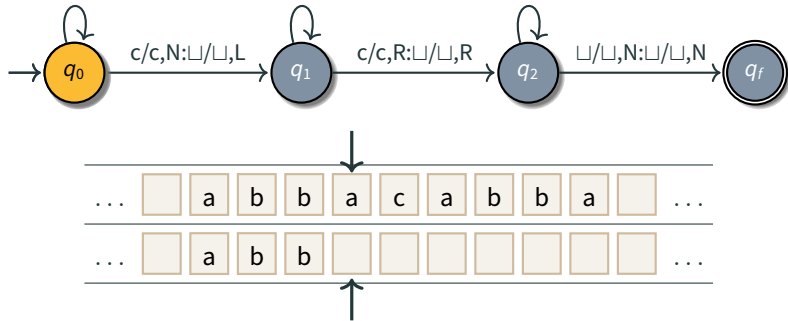
Wort bis "c" auf 2.
Band kopieren

MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$

$a/a, R: \sqcup/a, R$
 $b/b, R: \sqcup/b, R$

$c/c, N: a/a, L$
 $c/c, N: b/b, L$

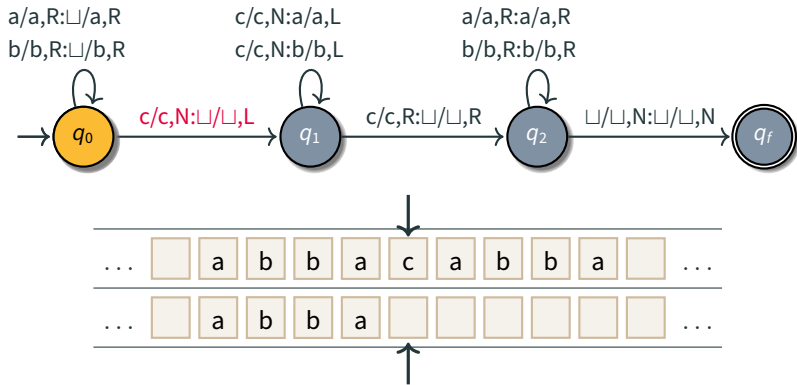
$a/a, R: a/a, R$
 $b/b, R: b/b, R$



q_0

Wort bis "c" auf 2.
Band kopieren

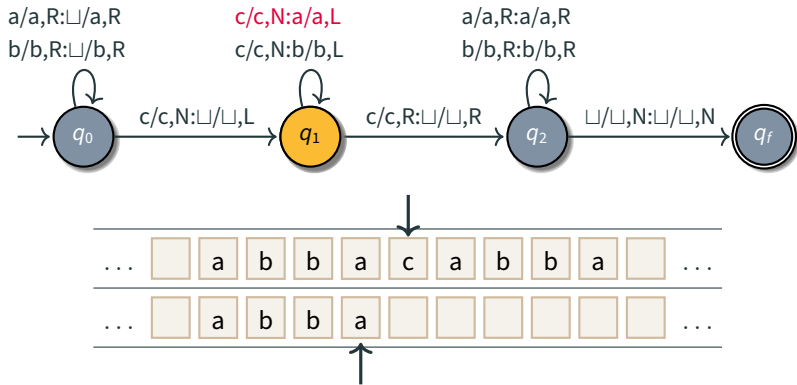
MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



q_0

Wort bis "c" auf 2.
Band kopieren

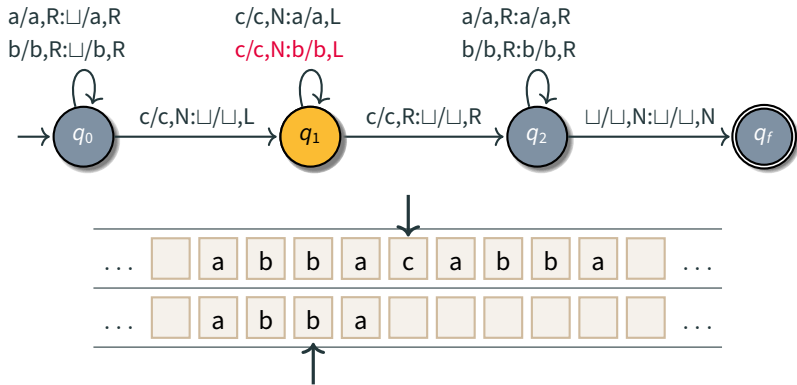
MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



q_1

Auf Band 2 zurück

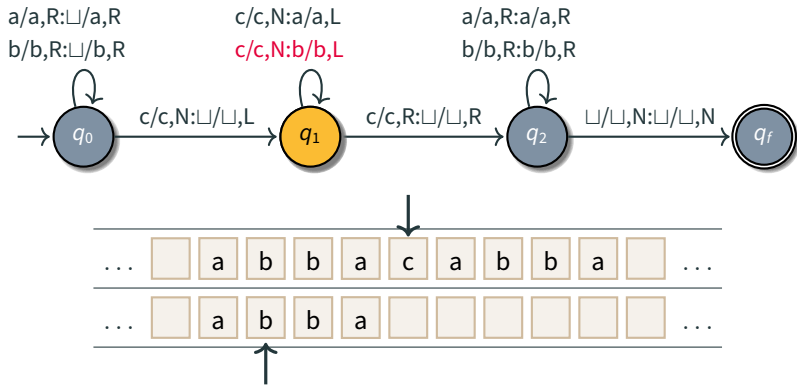
MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



q_1

Auf Band 2 zurück

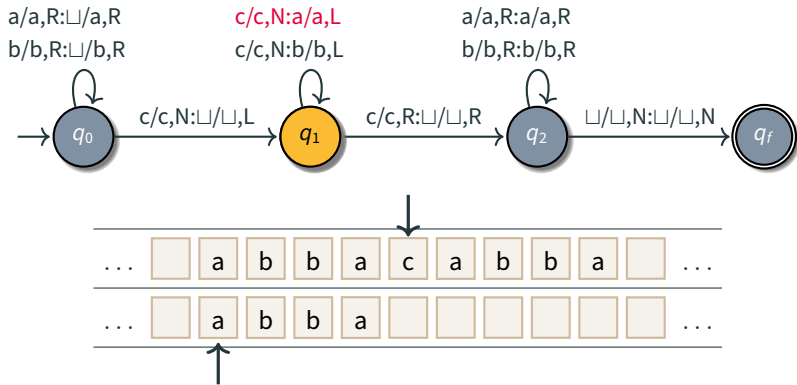
MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



q_1

Auf Band 2 zurück

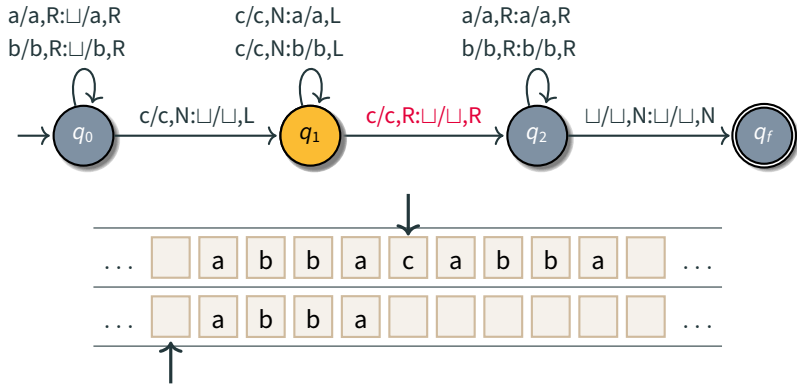
MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



q_1

Auf Band 2 zurück

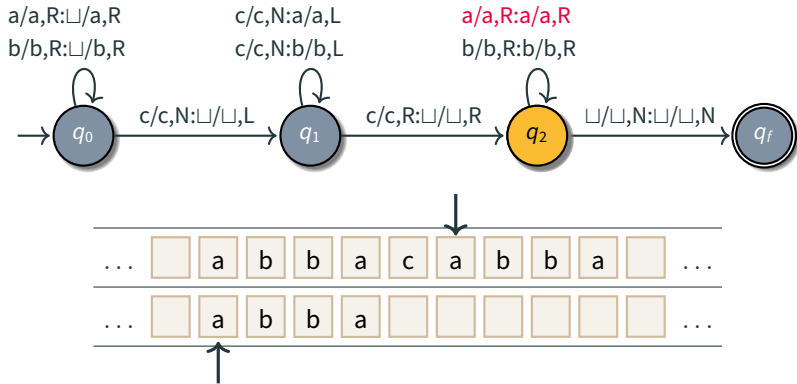
MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



q_1

Auf Band 2 zurück

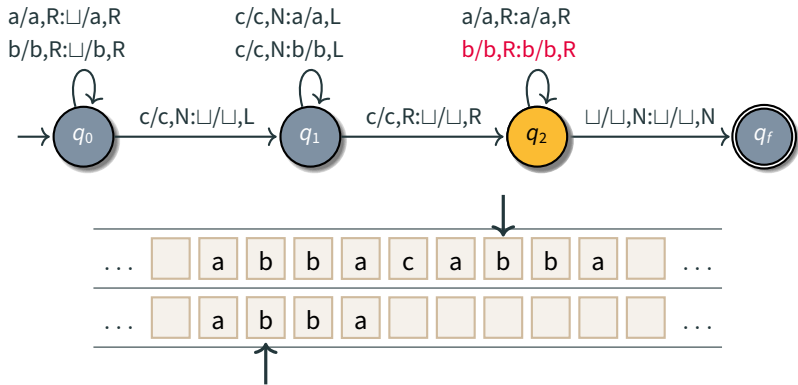
MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



q_2

Vergleich von Inhalt
Band 1 und 2

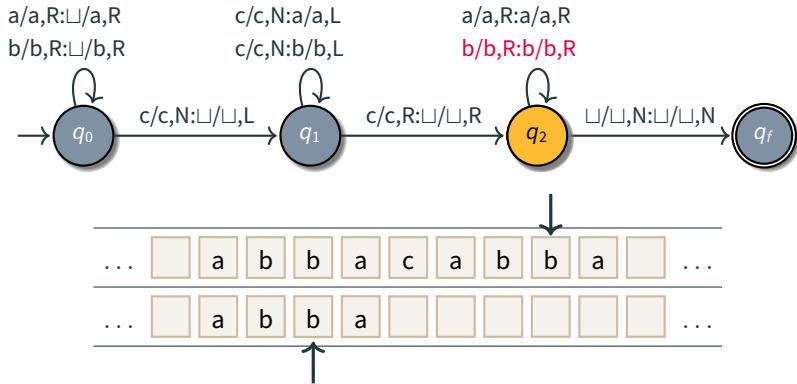
MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



q_2

Vergleich von Inhalt
Band 1 und 2

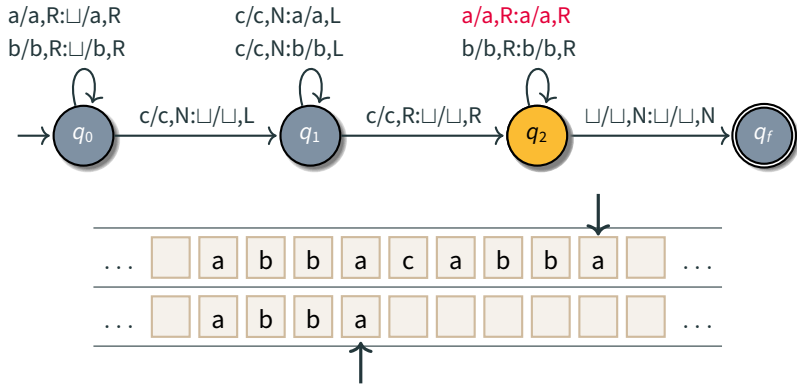
MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



q_2

Vergleich von Inhalt
Band 1 und 2

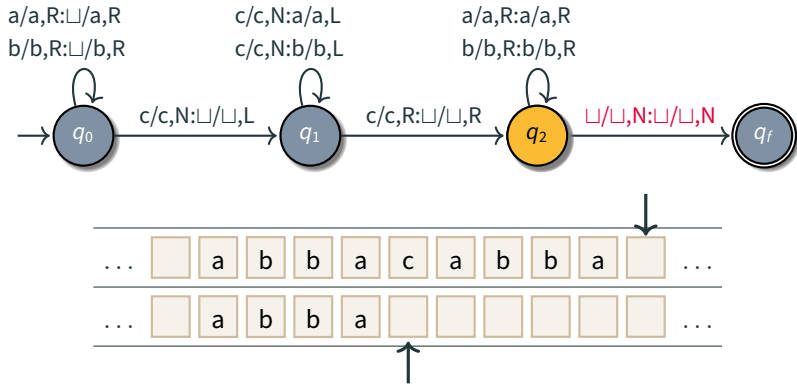
MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



q_2

Vergleich von Inhalt
Band 1 und 2

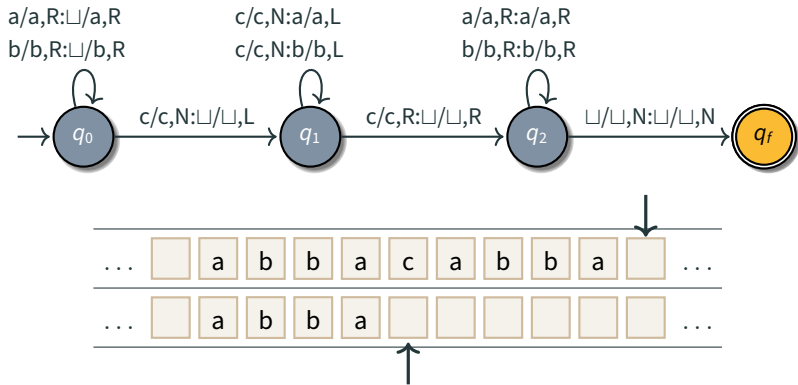
MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



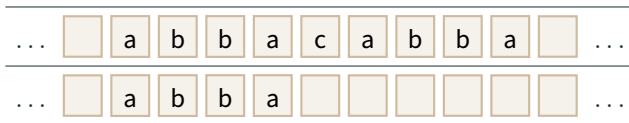
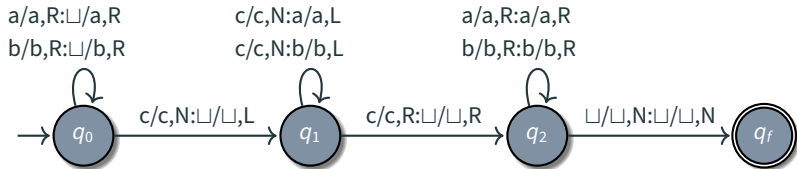
q_2

Vergleich von Inhalt
Band 1 und 2

MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



MEHRBAND-TM FÜR $\{wcw \mid w \in \{a, b\}^*\}$



- Wie könnte eine 1-Band TM die Sprache akzeptieren?

EINBAND-TM VS MEHRBAND-TM

Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.

$a/a, R: \sqcup/a, R$

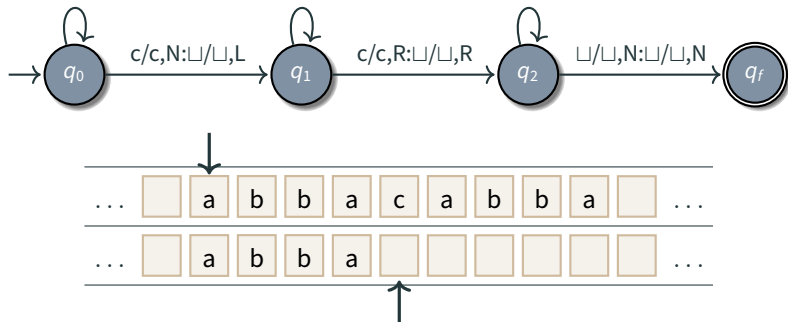
$c/c, N: a/a, L$

$a/a, R: a/a, R$

$b/b, R: \sqcup/b, R$

$c/c, N: b/b, L$

$b/b, R: b/b, R$



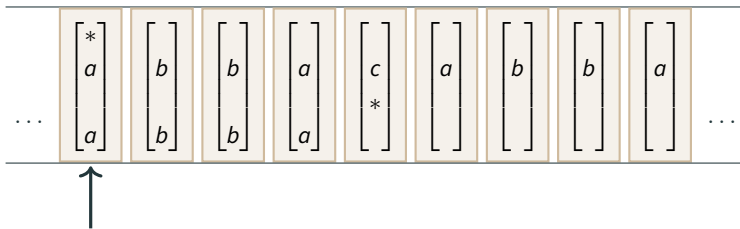
EINBAND-TM VS MEHRBAND-TM

Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.

$a/a, R: \sqcup/a, R$
 $b/b, R: \sqcup/b, R$

$c/c, N: a/a, L$
 $c/c, N: b/b, L$

$a/a, R: a/a, R$
 $b/b, R: b/b, R$



EINBAND-TM VS MEHRBAND-TM

Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.

$a/a, R: \square/a, R$

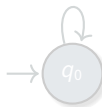
$b/b, R: \square/b, R$

$c/c, N: a/a, L$

$c/c, N: b/b, L$

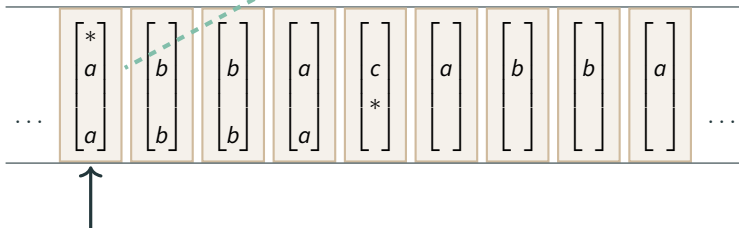
$a/a, R: a/a, R$

$b/b, R: b/b, R$



Bandalphabet erweitern

Ein Bandzeichen des neuen Alphabets kodiert Inhalt beider Bänder und ob jeweils ein Kopf "hier" steht.



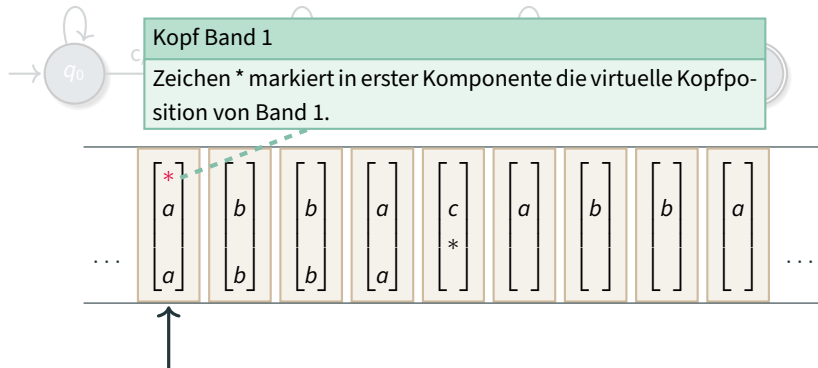
EINBAND-TM VS MEHRBAND-TM

Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.

$a/a, R: \square/a, R$
 $b/b, R: \square/b, R$

$c/c, N: a/a, L$
 $c/c, N: b/b, L$

$a/a, R: a/a, R$
 $b/b, R: b/b, R$



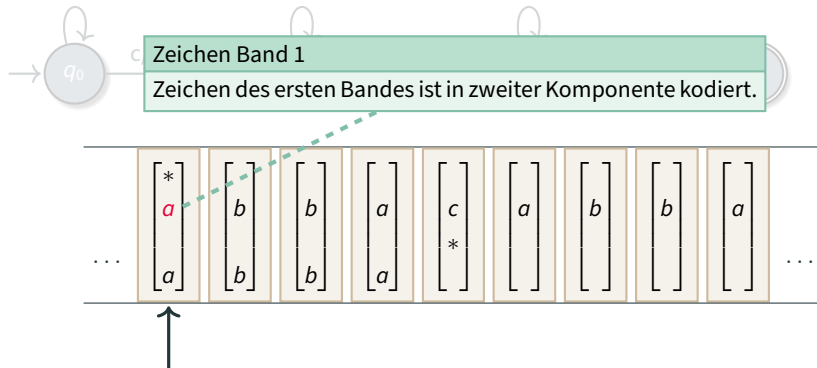
EINBAND-TM VS MEHRBAND-TM

Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.

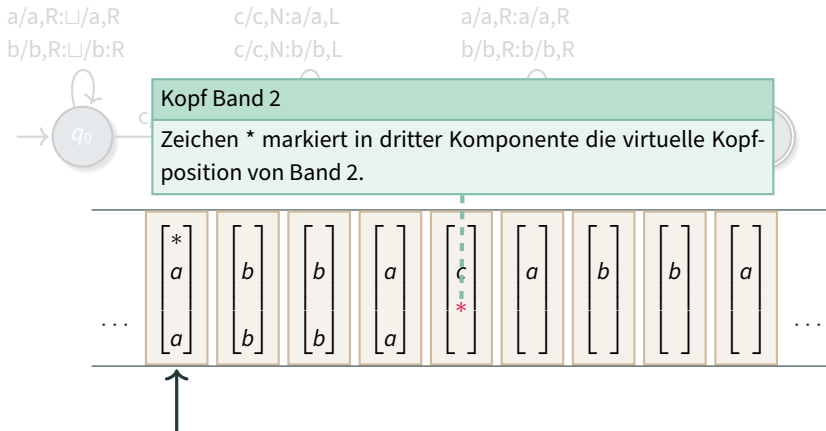
$a/a, R: \square/a, R$
 $b/b, R: \square/b, R$

$c/c, N: a/a, L$
 $c/c, N: b/b, L$

$a/a, R: a/a, R$
 $b/b, R: b/b, R$



Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.



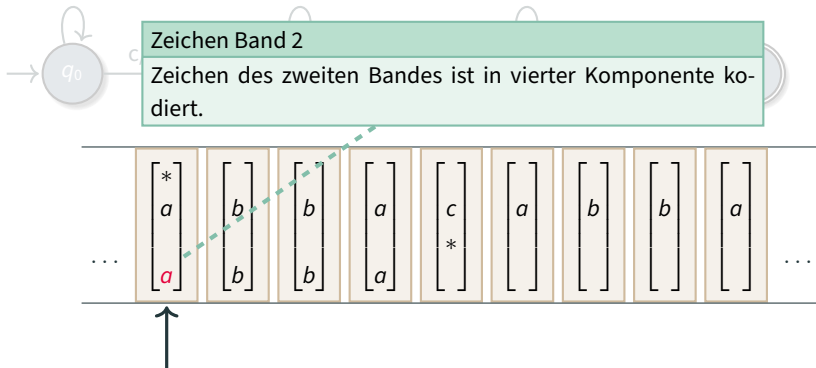
EINBAND-TM VS MEHRBAND-TM

Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.

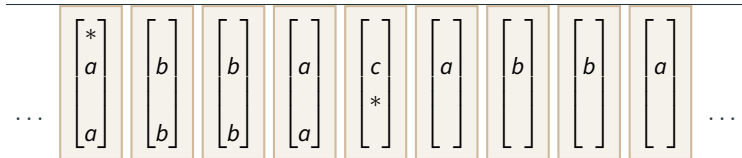
$a/a, R: \square/a, R$
 $b/b, R: \square/b, R$

$c/c, N: a/a, L$
 $c/c, N: b/b, L$

$a/a, R: a/a, R$
 $b/b, R: b/b, R$



Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.



- Maschine steht links von allen Markierungen
- Maschine läuft nach rechts bis alle Markierungen überschritten und ermittelt dabei welche Transition anzuwenden ist
- Maschine merkt sich in Zustand wie viele Markierungen links bzw. rechts vom Kopf stehen
- Maschine läuft nach links über alle Markierungen und wendet dabei Updates an

ALTERNATIVE IDEE DER SIMULATION EINER MEHRBAND-TM

Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.

$a/a, R: \sqcup/a, R$

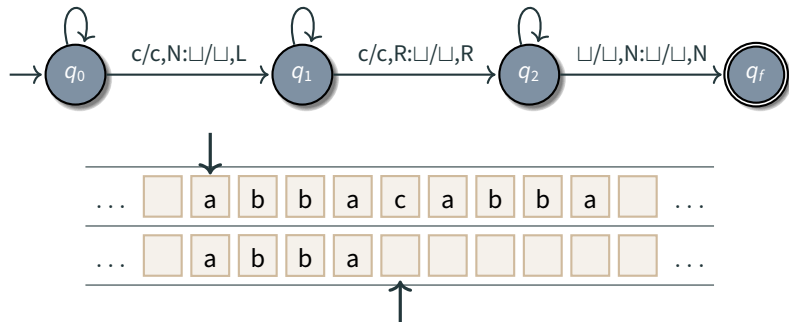
$c/c, N: a/a, L$

$a/a, R: a/a, R$

$b/b, R: \sqcup/b, R$

$c/c, N: b/b, L$

$b/b, R: b/b, R$



ALTERNATIVE IDEE DER SIMULATION EINER MEHRBAND-TM

Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.

$a/a, R: \sqcup/a, R$

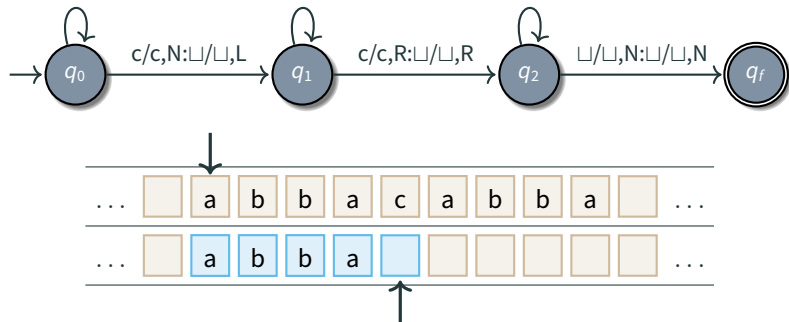
$c/c, N: a/a, L$

$a/a, R: a/a, R$

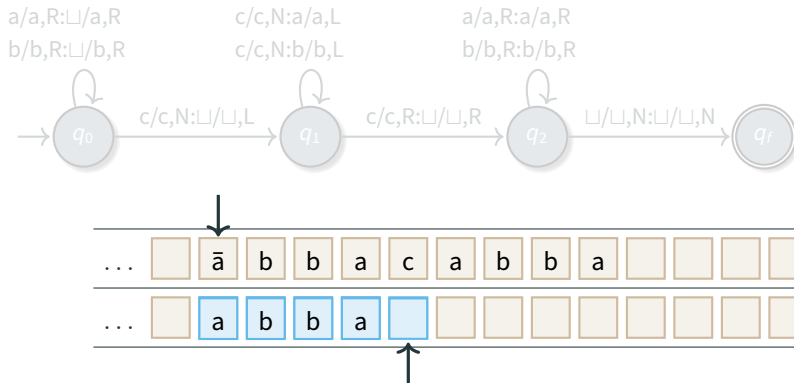
$b/b, R: \sqcup/b, R$

$c/c, N: b/b, L$

$b/b, R: b/b, R$

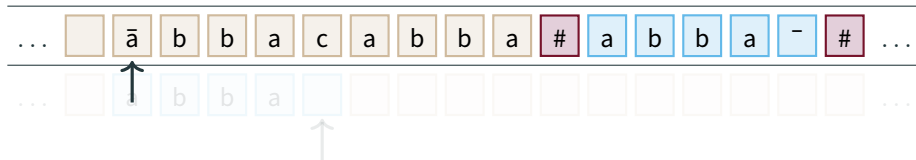
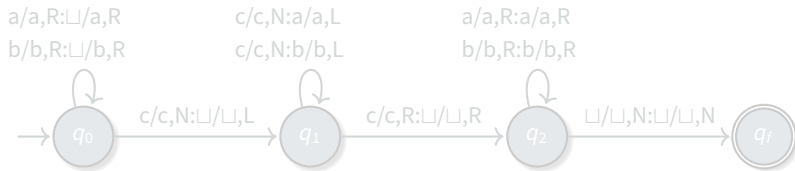


Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.



ALTERNATIVE IDEE DER SIMULATION EINER MEHRBAND-TM

Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.

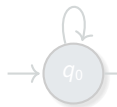


Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.

$a/a, R: \sqcup/a, R$
 $b/b, R: \sqcup/b, R$

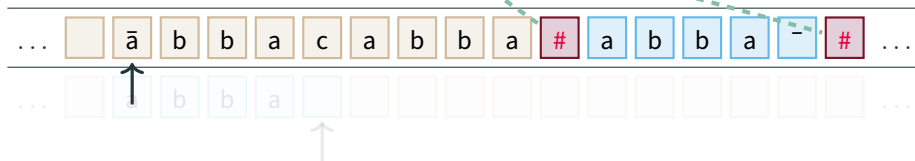
$c/c, N: a/a, L$
 $c/c, N: b/b, L$

$a/a, R: a/a, R$
 $b/b, R: b/b, R$



Trennzeichen

Inhalte der Bänder sind auf einem Band durch Trennzeichen getrennt

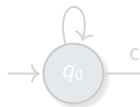


Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.

$a/a, R: \sqcup/a, R$
 $b/b, R: \sqcup/b, R$

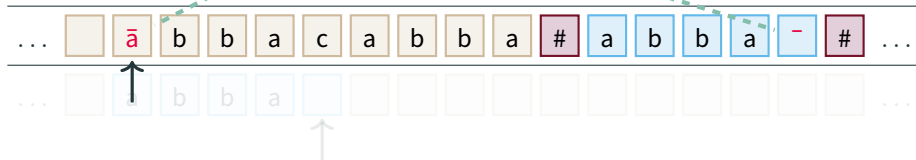
$c/c, N: a/a, L$
 $c/c, N: b/b, L$

$a/a, R: a/a, R$
 $b/b, R: b/b, R$



Kopfposition

Eintrag in Zelle wird mit $\bar{}$ markiert, falls Kopf auf Zelle steht. Bandalphabet wird entsprechend erweitert.

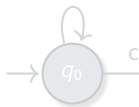


Jede Mehrband-TM kann durch eine Einband-TM simuliert werden.

$a/a, R: \sqcup/a, R$
 $b/b, R: \sqcup/b, R$

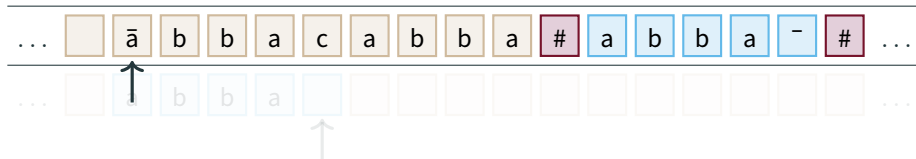
$c/c, N: a/a, L$
 $c/c, N: b/b, L$

$a/a, R: a/a, R$
 $b/b, R: b/b, R$



Band verschieben

Falls virtueller Kopf das Trennzeichen überschreitet,
muss das virtuelle Band um eine Position nach rechts
verschoben werden



Beobachtung

Prüfen, ob eine berechnete Lösung stimmt, ist häufig einfacher als die Lösung zu berechnen.

Beobachtung

Prüfen, ob eine berechnete Lösung stimmt, ist häufig einfacher als die Lösung zu berechnen.

Beispiel

Die Faktoren von 13352033 zu ermitteln ist eine Herausforderung.

Beobachtung

Prüfen, ob eine berechnete Lösung stimmt, ist häufig einfacher als die Lösung zu berechnen.

Beispiel

Die Faktoren von 13352033 zu ermitteln ist eine Herausforderung. Prüfen, dass $3407 \cdot 3919 = 13352033$ geht mit Papier und Bleistift ganz fix.

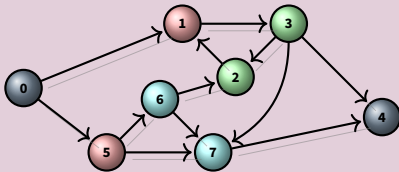
Beobachtung

Prüfen, ob eine berechnete Lösung stimmt, ist häufig einfacher als die Lösung zu berechnen.

Beispiel

Die Faktoren von 13352033 zu ermitteln ist eine Herausforderung. Prüfen, dass $3407 \cdot 3919 = 13352033$ geht mit Papier und Bleistift ganz fix.

Beispiel



Prüfen, ob es einen Weg gibt, der jeden Knoten genau einmal besucht, ist aufwändig.

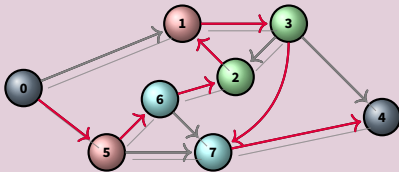
Beobachtung

Prüfen, ob eine berechnete Lösung stimmt, ist häufig einfacher als die Lösung zu berechnen.

Beispiel

Die Faktoren von 13352033 zu ermitteln ist eine Herausforderung. Prüfen, dass $3407 \cdot 3919 = 13352033$ geht mit Papier und Bleistift ganz fix.

Beispiel

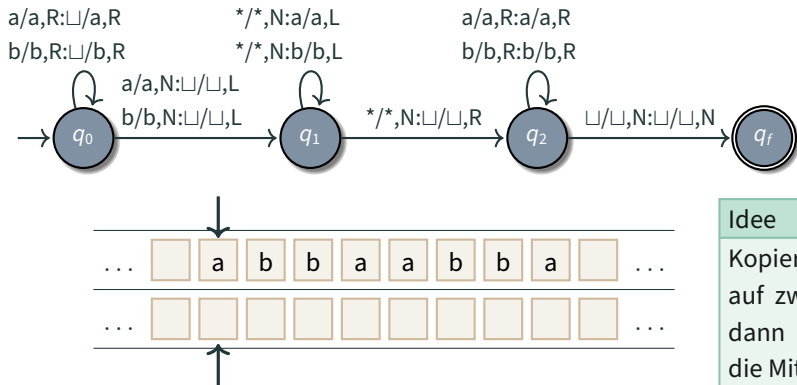


Prüfen, ob es einen Weg gibt, der jeden Knoten genau einmal besucht, ist aufwändig. Einen gegebenen Weg zu prüfen, ist einfach.

DIE SPRACHE $\{ww \mid w \in \{a,b\}^+\}$

- Linker und rechter Wortteil müssen übereinstimmen.
- Problem: Wir müssen wissen, wo die Mitte ist.

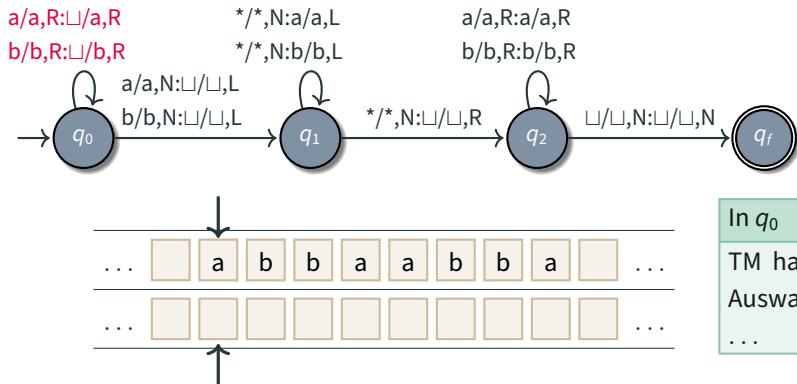
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Idee

Kopiere erste Hälfte der Eingabe auf zweites Band und vergleiche dann mit die beiden Teile. Finde die Mitte durch Raten.

NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$

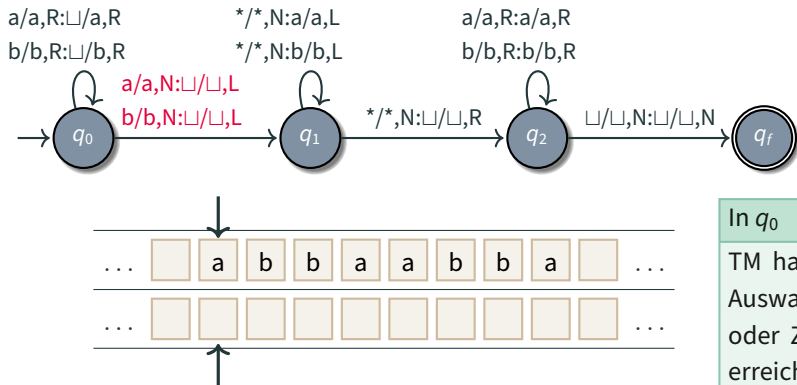


In q_0

TM hat die nichtdeterministische
Auswahl zwischen Wort kopieren

...

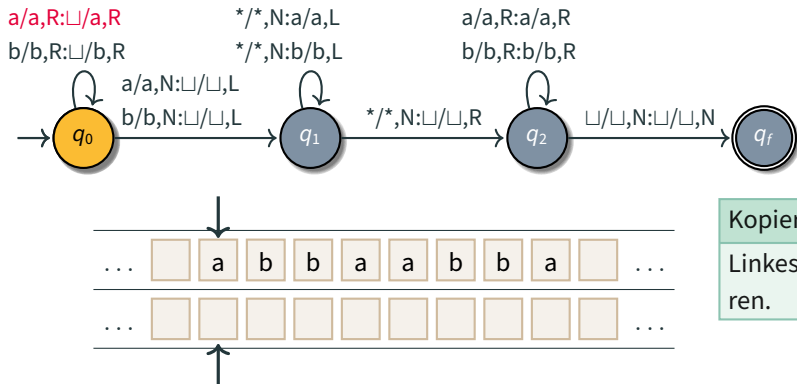
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



In q_0

TM hat die nichtdeterministische Auswahl zwischen Wort kopieren oder Zustand wechseln weil Mitte erreicht.

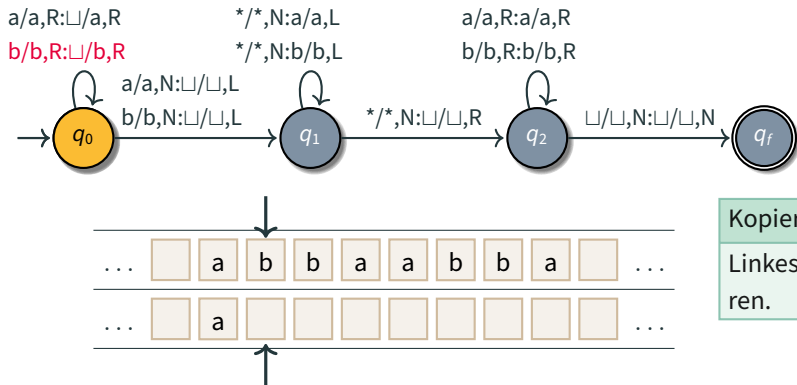
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Kopieren

Linkes Teilwort auf Band 2 kopieren.

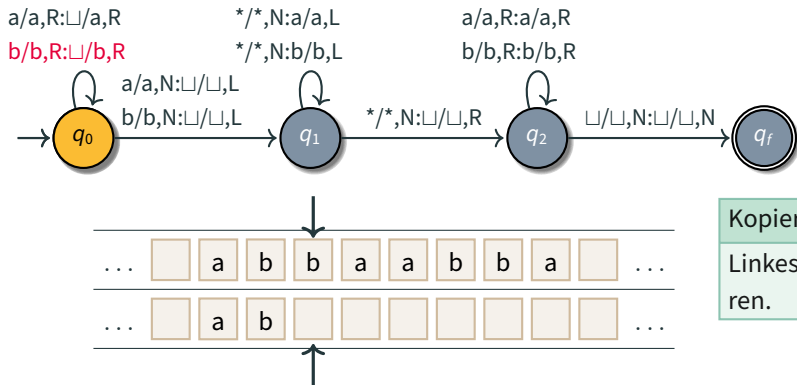
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Kopieren

Linkes Teilwort auf Band 2 kopieren.

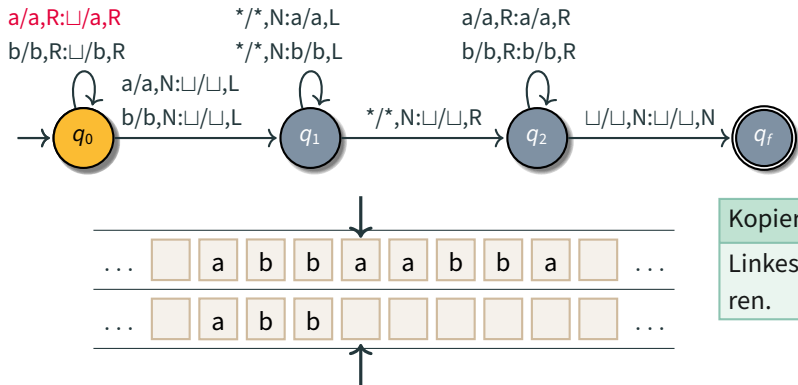
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Kopieren

Linkes Teilwort auf Band 2 kopieren.

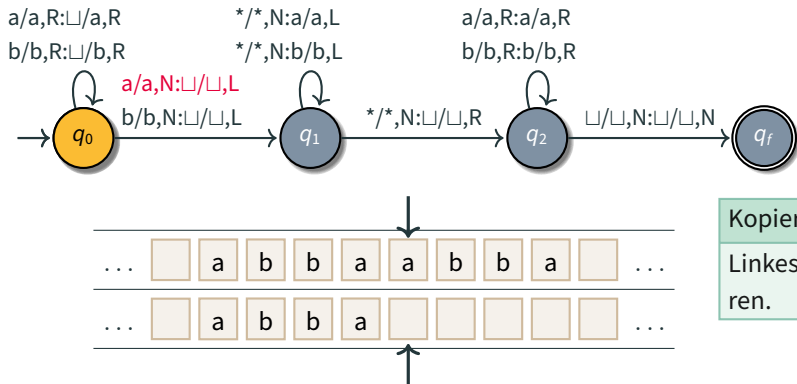
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Kopieren

Linkes Teilwort auf Band 2 kopieren.

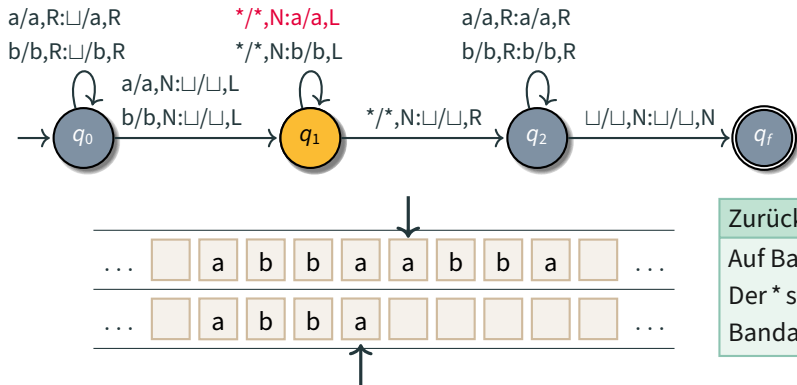
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Kopieren

Linkes Teilwort auf Band 2 kopieren.

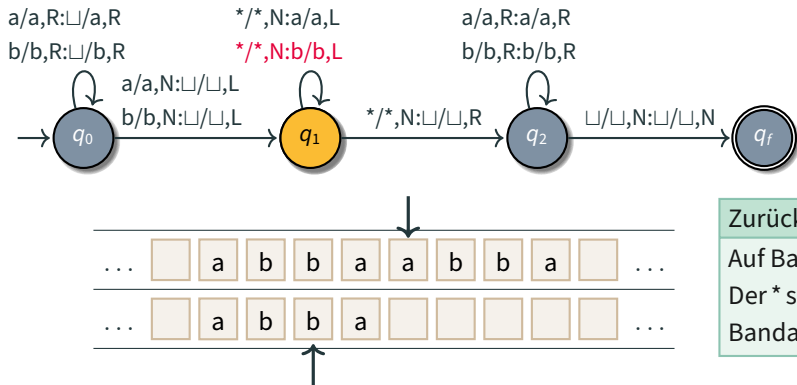
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Zurück

Auf Band 2 zurück nach links.
Der * sei ein beliebiges Zeichen des Bandalphabets.

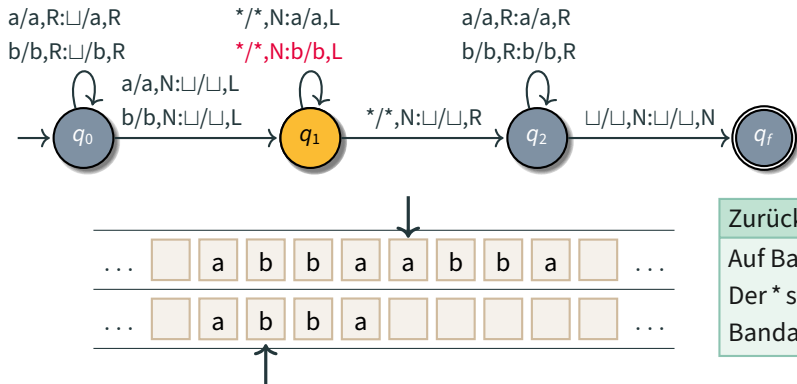
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Zurück

Auf Band 2 zurück nach links.
Der $\ast$ sei ein beliebiges Zeichen des Bandalphabets.

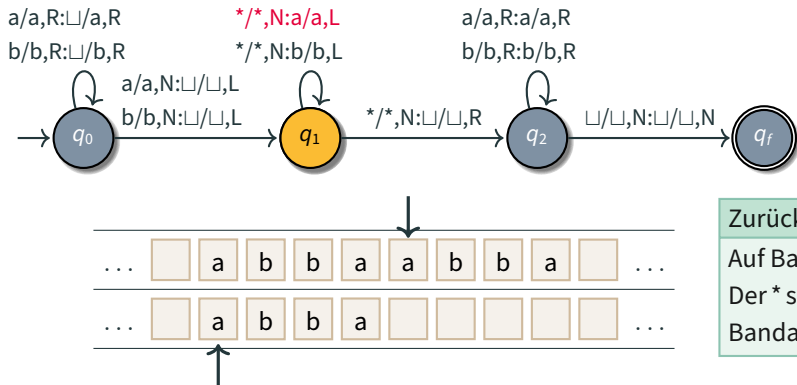
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Zurück

Auf Band 2 zurück nach links.
Der * sei ein beliebiges Zeichen des Bandalphabets.

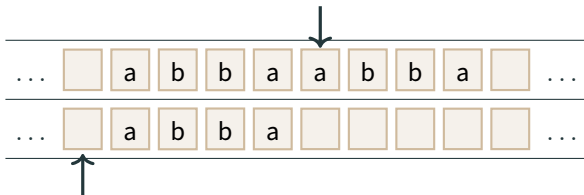
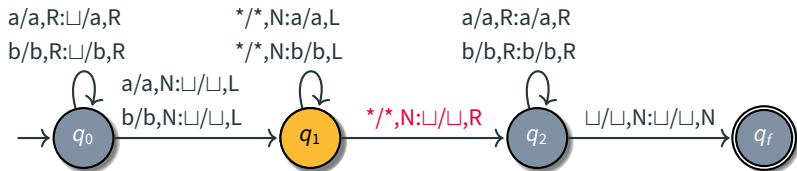
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Zurück

Auf Band 2 zurück nach links.
Der $*$ sei ein beliebiges Zeichen des
Bandalphabets.

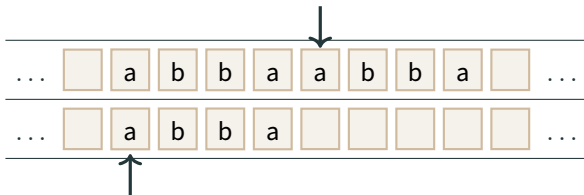
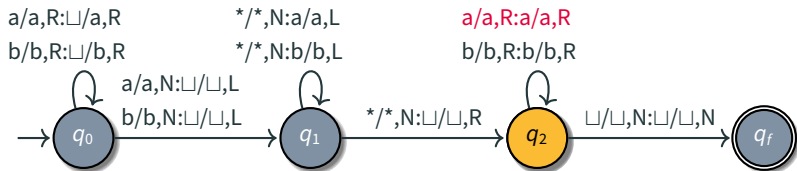
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



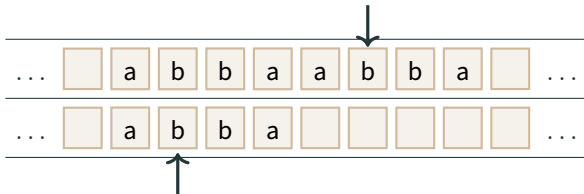
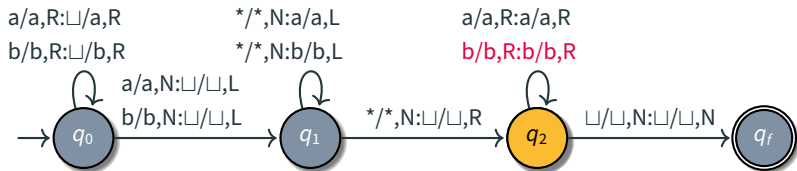
Zurück

Auf Band 2 zurück nach links.
Der * sei ein beliebiges Zeichen des
Bandalphabets.

NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



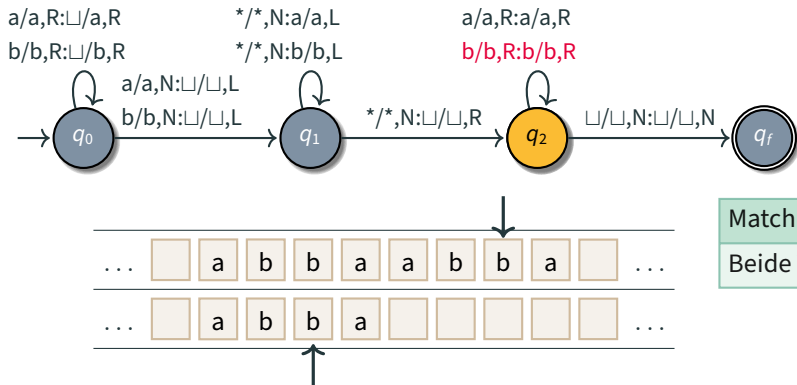
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Matching

Beide Teilwörter vergleichen.

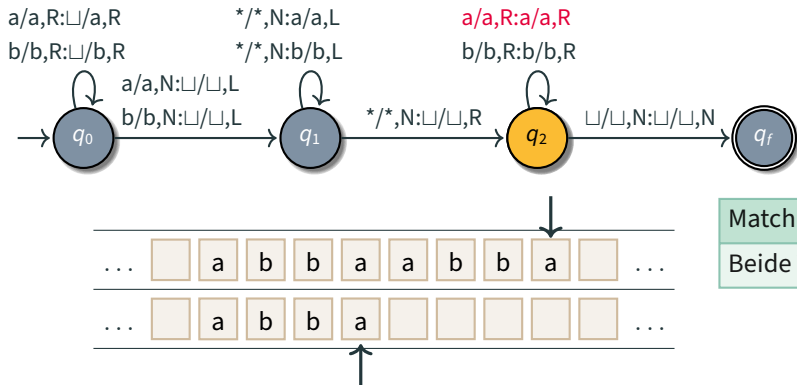
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Matching

Beide Teilwörter vergleichen.

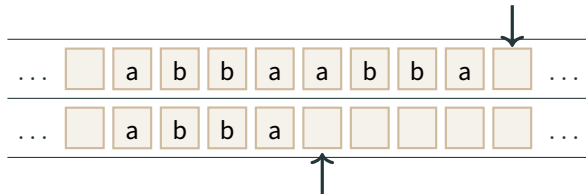
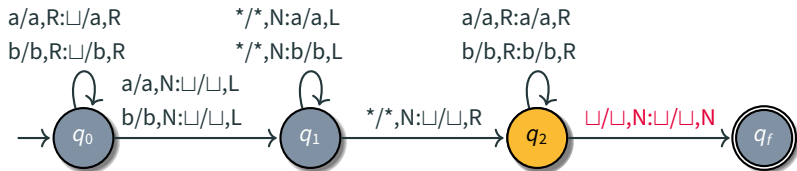
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Matching

Beide Teilwörter vergleichen.

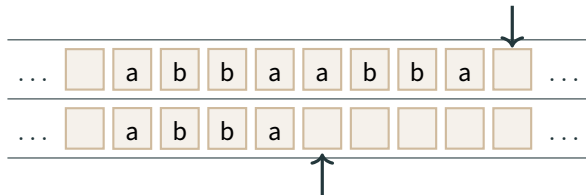
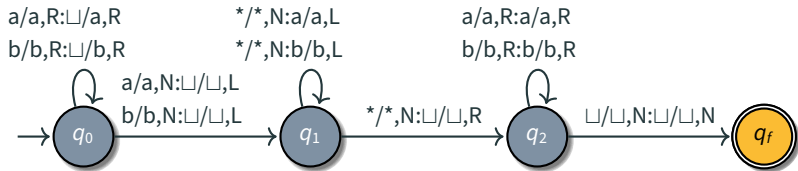
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Matching

Beide Teilwörter vergleichen.

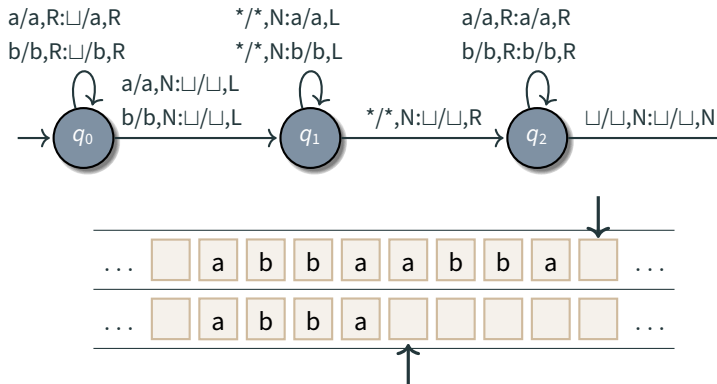
NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Akzeptanz

Eine nichtdeterministische Turing-Maschine akzeptiert ein Wort, wenn es einen Berechnungspfad gibt, der zu einem Endzustand führt.

NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$

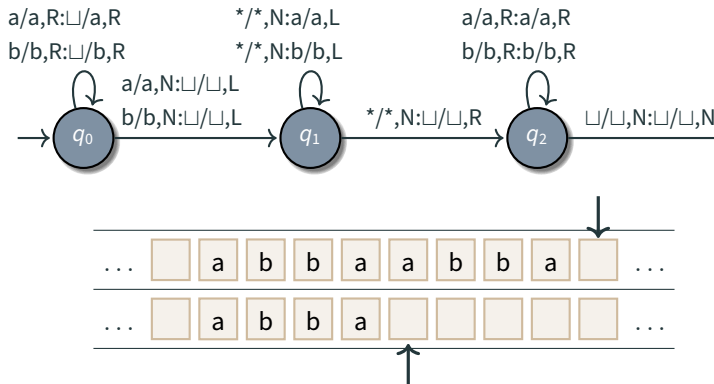


Guess and Check

Typisches Vorgehen:

1. Raten einer Lösung (z.B. Mitte raten)
2. Prüfen, ob richtig geraten wurde (Matching der Wortteile).

NICHTDETERMINISTISCHE TURING-MASCHINE FÜR $\{ww \mid w \in \{a, b\}^+\}$



Achtung

Auf das Prüfen kann nicht verzichtet werden, weil die TM nur in einen Endzustand gehen darf, wenn das Wort auch in der Sprache ist.

Satz

Jede nichtdeterministische TM kann durch eine deterministische TM simuliert werden.

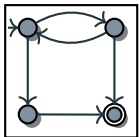
Wie?

Satz

Jede nichtdeterministische TM kann durch eine deterministische TM simuliert werden.

- Idee: Alle Konfigurationen der nichtdeterministischen TM (Tripel aus Zustand, Kopfposition und Bandinhalt) durch deterministische TM in Breitensuche durchgehen
- Tiefensuche funktioniert nicht, falls TM in Endlosschleife gerät
- Breitensuche mit Hilfe einer Queue:
 1. Neue Konfiguration aus Queue rausholen
 2. Falls Endzustand erreicht - Stoppen und akzeptieren
 3. Sonst: Alle möglichen Nachfolger bestimmen und in Queue stecken

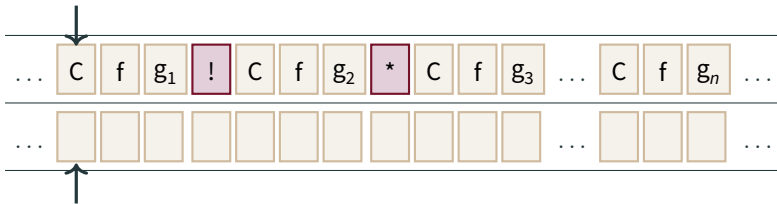
SIMULATION EINER NICHTDETERMINISTISCHEN TM



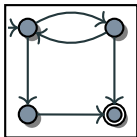
Simulator

Simulator

Deterministische 2-Band-TM simuliert
nichtdeterministische 1-Band-TM.



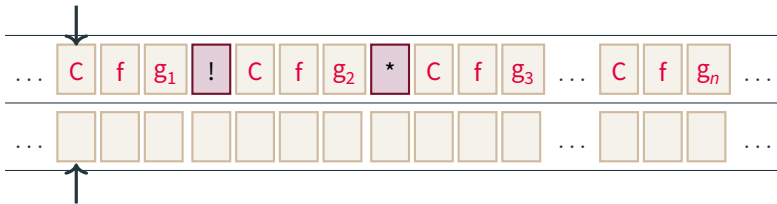
SIMULATION EINER NICHTDETERMINISTISCHEN TM



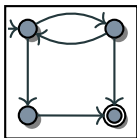
Simulator

Band 1

Queue mit Konfigurationen der simulierten nichtdeterministischen TM.



SIMULATION EINER NICHTDETERMINISTISCHEN TM

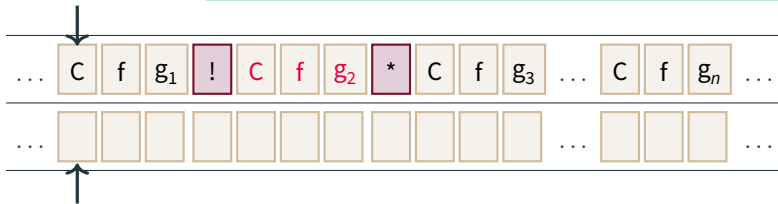


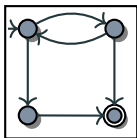
Simulator

Konfiguration Cfg_i : Bandinhalt, Kopfposition, Zustand

Beispiel: $xabbq_1bbccc$

- Zustand q_1
- Links von Kopf $xabb$
- Rechts bzw. unter Kopf $bbccc$

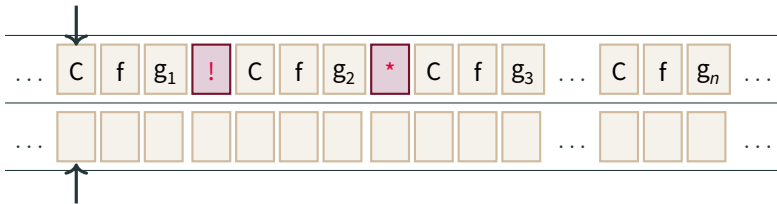




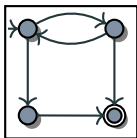
Simulator

Trennzeichen

- ! markiert Kopf der Queue und damit nächste zu bearbeitende Konfiguration.
- * trennt Konfigurationen.



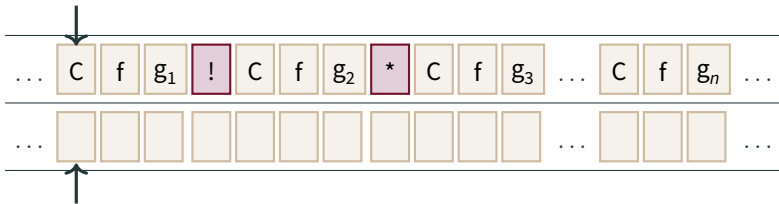
SIMULATION EINER NICHTDETERMINISTISCHEN TM



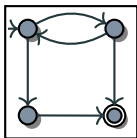
Simulator

Band 2

Arbeitsband



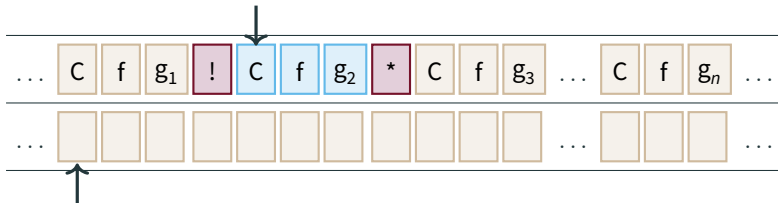
SIMULATION EINER NICHTDETERMINISTISCHEN TM



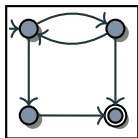
Simulator

Bearbeite nächste Konfiguration

1. Prüfe, ob Konfiguration Endzustand enthält
2. Falls nein, kopiere Konfiguration auf Arbeitsband



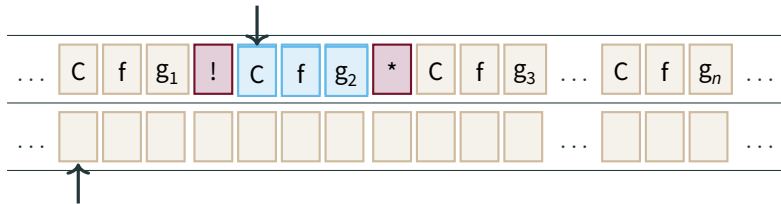
SIMULATION EINER NICHTDETERMINISTISCHEN TM



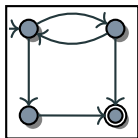
Simulator

Bearbeite nächste Konfiguration

1. Prüfe, ob Konfiguration Endzustand enthält
2. Falls nein, kopiere Konfiguration auf Arbeitsband



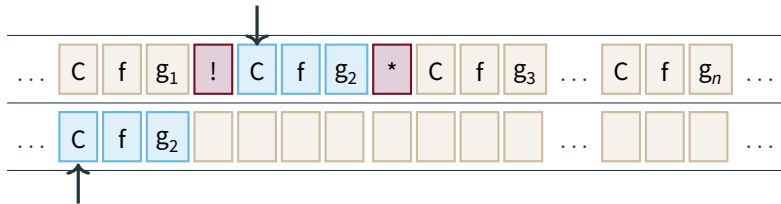
SIMULATION EINER NICHTDETERMINISTISCHEN TM

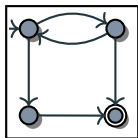


Simulator

Bearbeite nächste Konfiguration

1. Prüfe, ob Konfiguration Endzustand enthält
2. Falls nein, kopiere Konfiguration auf Arbeitsband

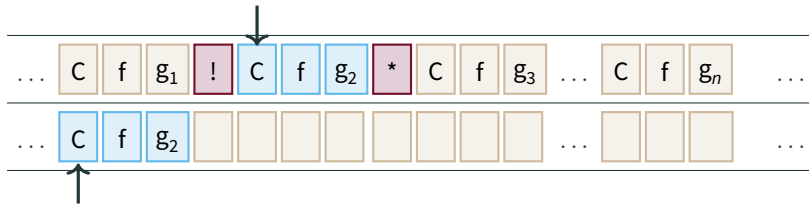


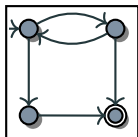


Simulator

Bearbeite nächste Konfiguration

1. Prüfe, ob Konfiguration Endzustand enthält
2. Falls nein, kopiere Konfiguration auf Arbeitsband
3. Nachfolgekongfigurationen erzeugen und in Queue anhängen

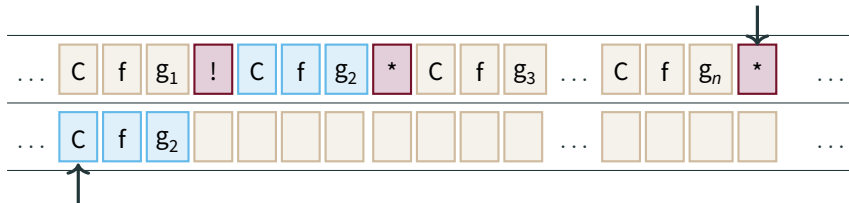


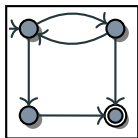


Simulator

Bearbeite nächste Konfiguration

1. Prüfe, ob Konfiguration Endzustand enthält
2. Falls nein, kopiere Konfiguration auf Arbeitsband
3. Nachfolgekongfigurationen erzeugen und in Queue anhängen

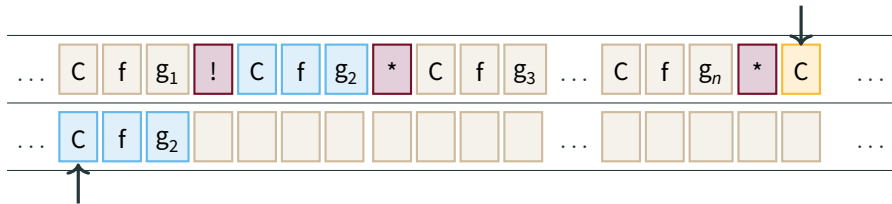


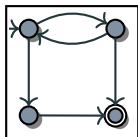


Simulator

Bearbeite nächste Konfiguration

1. Prüfe, ob Konfiguration Endzustand enthält
2. Falls nein, kopiere Konfiguration auf Arbeitsband
3. Nachfolgekongfigurationen erzeugen und in Queue anhängen

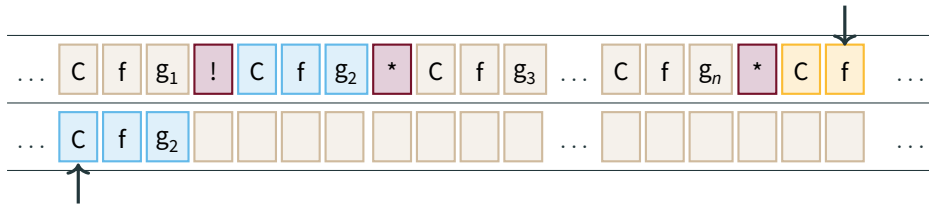


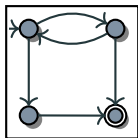


Simulator

Bearbeite nächste Konfiguration

1. Prüfe, ob Konfiguration Endzustand enthält
2. Falls nein, kopiere Konfiguration auf Arbeitsband
3. Nachfolgekongfigurationen erzeugen und in Queue anhängen

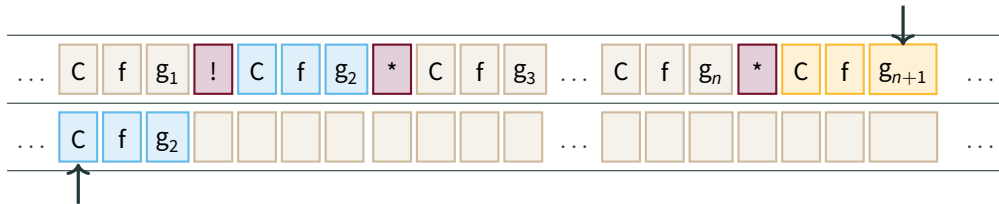


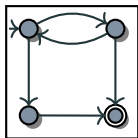


Simulator

Bearbeite nächste Konfiguration

1. Prüfe, ob Konfiguration Endzustand enthält
2. Falls nein, kopiere Konfiguration auf Arbeitsband
3. Nachfolgekongfigurationen erzeugen und in Queue anhängen

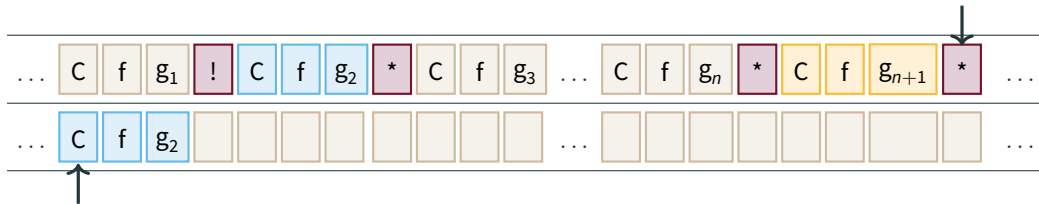


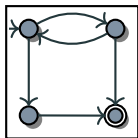


Simulator

Bearbeite nächste Konfiguration

1. Prüfe, ob Konfiguration Endzustand enthält
2. Falls nein, kopiere Konfiguration auf Arbeitsband
3. Nachfolgekonfigurationen erzeugen und in Queue anhängen

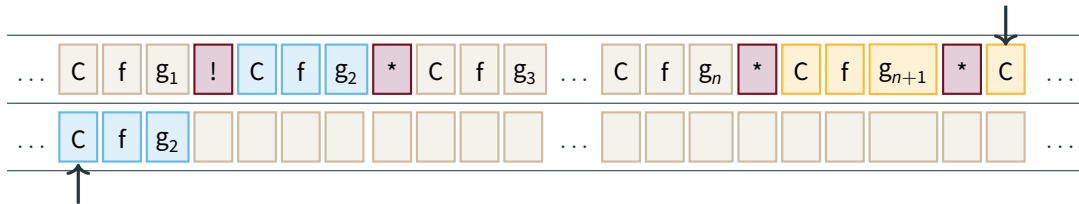


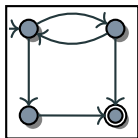


Simulator

Bearbeite nächste Konfiguration

1. Prüfe, ob Konfiguration Endzustand enthält
2. Falls nein, kopiere Konfiguration auf Arbeitsband
3. Nachfolgekongfigurationen erzeugen und in Queue anhängen

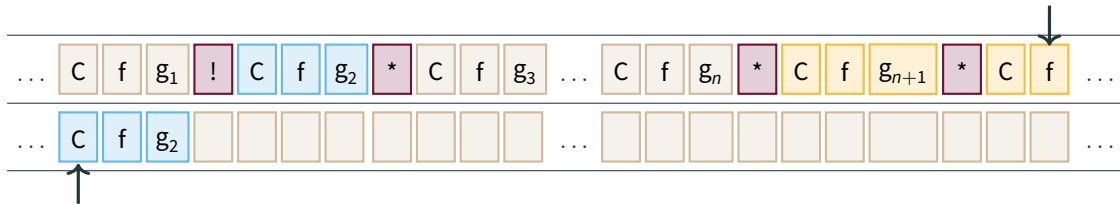


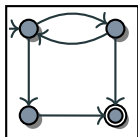


Simulator

Bearbeite nächste Konfiguration

1. Prüfe, ob Konfiguration Endzustand enthält
2. Falls nein, kopiere Konfiguration auf Arbeitsband
3. Nachfolgekongfigurationen erzeugen und in Queue anhängen

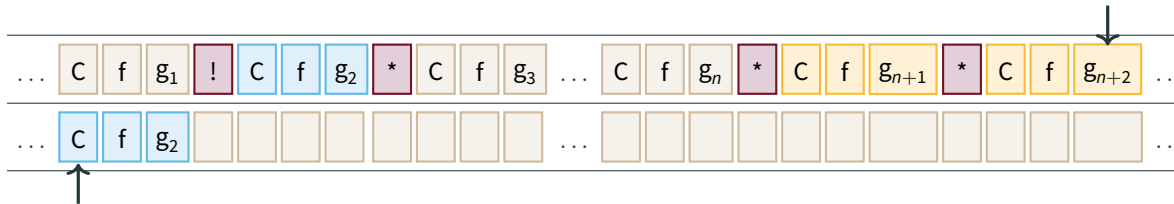


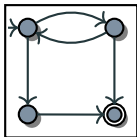


Simulator

Bearbeite nächste Konfiguration

1. Prüfe, ob Konfiguration Endzustand enthält
2. Falls nein, kopiere Konfiguration auf Arbeitsband
3. Nachfolgekongfigurationen erzeugen und in Queue anhängen

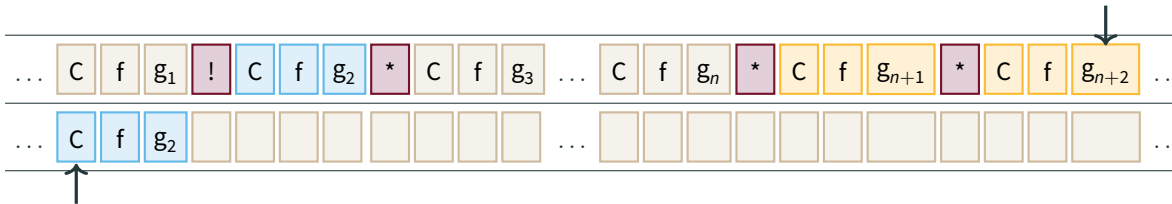


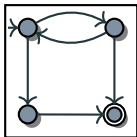


Simulator

Bearbeite nächste Konfiguration

4. Marker ! verschieben, Arbeitsband löschen

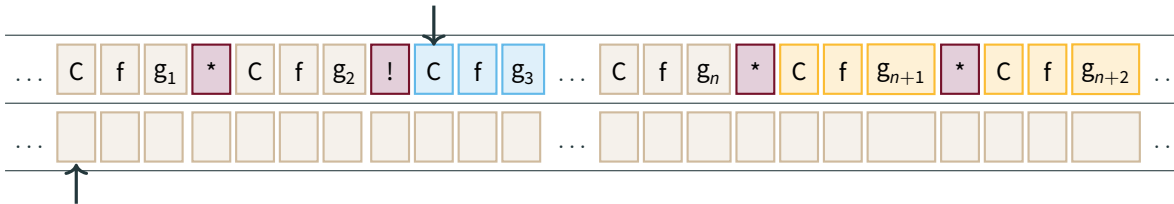


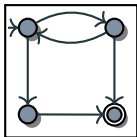


Simulator

Bearbeite nächste Konfiguration

4. Marker ! verschieben, Arbeitsband löschen





Simulator

Akzeptanz

1. Falls Endzustand in Konfiguration gefunden wird, akzeptiert TM sofort.
2. Falls alle Konfigurationen erfolglos abgearbeitet wurden, hält TM ohne zu akzeptieren.

